

Training School on Scientific Machine Learning for Digital Twins

Galerkin Projection for PDEs

Spectral methods, finite elements and POD-Galerkin

From high-fidelity physics to real-time digital twins

M. A. Mendez

von Karman Institute for Fluid Dynamics, Belgium

mendez@vki.ac.be

1. Introduction: why we need cheap models
2. Approximation spaces, inner products and projection
Tutorial 1: approximating a function
3. Galerkin projection of a PDE
Tutorial 2: one PDE, three discretisations
4. POD and POD-Galerkin reduced-order models
Tutorial 3: POD and a POD-Galerkin ROM
5. Synthesis

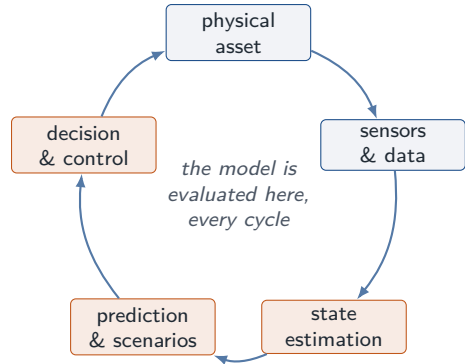
Each tutorial is run inside the section it belongs to. *(every line above is clickable, and the small line above each frame title comes back here)*

What does a digital twin have to compute?

Repeatedly, and often in real time:

- state estimation
- short-term prediction
- scenario evaluation
- parameter and fault estimation
- optimisation and control
- uncertainty propagation

Every one of these needs *many* model evaluations.



The cost mismatch

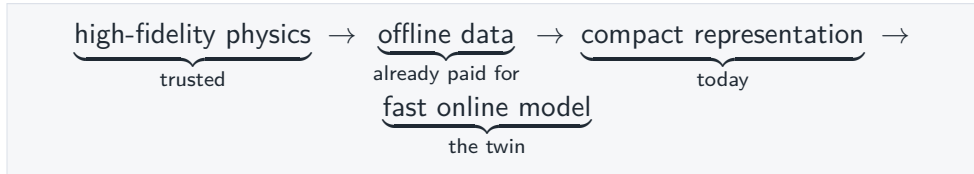
high-fidelity PDE model	seconds to hours per run
optimisation / UQ / MPC loop	10^2 – 10^5 runs
available wall-clock time	seconds

How can we **retain the information** contained in a high-fidelity physical model while making repeated simulations **dramatically cheaper**?

This is the question the whole lecture is built around.

Reuse: we are not starting from scratch

By the time the twin goes online we typically already own previous simulations, experimental campaigns, parameter sweeps, calibration studies and historical operating data.



Offline (*hours or days, once*)
simulations, database, basis, projection

Online (*milliseconds, constantly*)
evolve n_b coefficients, estimate, control

From a field to a handful of numbers

$$\underbrace{u(x, t)}_{\text{field: } n_x=10^5-10^8} \longrightarrow \underbrace{\mathbf{a}(t) = [a_1(t), \dots, a_{n_b}(t)]^\top}_{n_b=\mathcal{O}(10)}$$

Two questions, and they are the backbone of the lecture:

1. **Which** finite-dimensional space do we represent u in?
2. **How** do we make the governing PDE determine the $a_n(t)$?

The second question is answered once, in Section 3, and the answer is the same for every method in this lecture.

The common pipeline, and four ways to fill it



Finite differences approximate the *derivatives*

Spectral methods a few *global* basis functions

Finite elements a flexible *local* basis

POD let *previous solutions* choose the basis

Plan, and how we will work

- 2. approximation spaces, inner products, projection (+ *Tutorial 1*)
- 3. Galerkin projection of a PDE: FD, spectral, FEM (+ *Tutorial 2*)
- 4. POD and POD-Galerkin reduced-order models (+ *Tutorial 3*)
- 5. synthesis

Each tutorial runs **inside** its section. We derive, then implement, then look at the result before moving on.

Main question: how do we construct finite-dimensional approximation spaces, and how do we **project** the PDE onto them?

Section 2

Approximation spaces, inner products and projection

- 2.1 Warm-up in $\mathbb{R}^3$
- 2.2 From vectors to functions
- 2.3 Tutorial 1: approximating a function

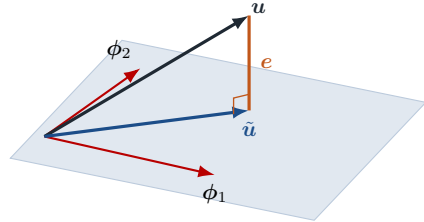
The simplest version of today's problem

Question. Given $\mathbf{u} \in \mathbb{R}^3$ and a plane through the origin, what is the best approximation of $\mathbf{u}$ *inside* that plane?

$$\mathcal{V}_r = \text{span}\{\phi_1, \phi_2\}$$

$$\tilde{\mathbf{u}} = \sum_{n=1}^{n_b} c_n \phi_n$$

$$\mathbf{e} = \mathbf{u} - \tilde{\mathbf{u}}$$



- **best** means smallest $\|\mathbf{e}\|$
- the unknowns are the n_b numbers c_n

Setting up the minimisation

Collect the basis in a matrix and the coefficients in a vector:

$$\Phi = [\phi_1 \ \phi_2 \ \cdots \ \phi_{n_b}] \in \mathbb{R}^{n_x \times n_b}, \quad \mathbf{c} \in \mathbb{R}^{n_b}, \quad \tilde{\mathbf{u}} = \Phi \mathbf{c}.$$

The error and the cost function are then

$$\mathbf{e}(\mathbf{c}) = \mathbf{u} - \Phi \mathbf{c}, \quad J(\mathbf{c}) = \|\mathbf{e}\|^2 = \mathbf{e}^\top \mathbf{e}.$$

Over to you

What are the shapes of $\partial J / \partial \mathbf{e}$ and $\partial \mathbf{e} / \partial \mathbf{c}$, and in which order do they multiply?

The orthogonality condition, in vector form

Apply the chain rule, treating J as a scalar function of the vector e , and e as a vector function of c :

$$\underbrace{\frac{\partial J}{\partial c}}_{1 \times n_b} = \underbrace{\frac{\partial J}{\partial e}}_{1 \times n_x} \underbrace{\frac{\partial e}{\partial c}}_{n_x \times n_b} = (2e^\top)(-\Phi) = -2e^\top \Phi$$

Setting the gradient to zero gives

$$\Phi^\top e = 0 \quad \Longleftrightarrow \quad \langle \phi_m, e \rangle = 0, \quad m = 1, \dots, n_b.$$

- row m of $\Phi^\top e$ is the inner product of ϕ_m with the error
- geometrically, $\tilde{u}$ is the foot of the perpendicular
- no correction inside the space can reduce the error further

Normal equations and the Gram matrix

Substitute $e = u - \Phi c$ into $\Phi^\top e = 0$:

$$\Phi^\top (u - \Phi c) = 0 \quad \implies \quad \underbrace{\Phi^\top \Phi}_A c = \underbrace{\Phi^\top u}_b$$

Entry by entry, $A_{mn} = \langle \phi_m, \phi_n \rangle$ and $b_m = \langle \phi_m, u \rangle$.

generic basis	solve $A c = b$
orthogonal basis	A diagonal, $c_n = \langle \phi_n, u \rangle / \ \phi_n\ ^2$
orthonormal basis	$A = I$, $c_n = \langle \phi_n, u \rangle$

(The same matrix returns in Section 3 as the mass matrix of the finite element method, and equals I for orthonormal spectral bases and for POD modes.)

Solving the normal equations

$\mathbf{A}$ is **symmetric** and, for a linearly independent basis, **positive definite**:

$$A_{mn} = \langle \phi_m, \phi_n \rangle = A_{nm}, \quad \mathbf{z}^\top \mathbf{A} \mathbf{z} = \left\| \sum_n z_n \phi_n \right\|^2 > 0 \quad \text{for } \mathbf{z} \neq \mathbf{0}.$$

These two properties are what the **Cholesky** factorisation requires:

$$\mathbf{A} = \mathbf{L} \mathbf{L}^\top, \quad \mathbf{L} \text{ lower triangular}, \quad \underbrace{\mathbf{L} \mathbf{y} = \mathbf{b}}_{\text{forward}} \text{ then } \underbrace{\mathbf{L}^\top \mathbf{c} = \mathbf{y}}_{\text{back}}.$$

- each sweep is a substitution, $\mathcal{O}(n_b^2)$; the factorisation is the only $\mathcal{O}(n_b^3)$ step
- half the cost and half the storage of LU , no pivoting, backward stable
- it fails exactly when $\mathbf{A}$ stops being numerically positive definite, which signals that the basis has become dependent

(In Python: `numpy.linalg.cholesky` for the factor, then `scipy.linalg.solve_triangular` for each sweep.)

Why orthogonality is worth the effort

- **decoupling**: each c_n computed independently of the others
- **hierarchy**: adding ϕ_{n_b+1} leaves $c_1, \dots, c_{n_b}$ untouched (*false for a generic basis*)
- **stability**: no ill-conditioned system to solve
- **Pythagoras**: $\|\mathbf{u}\|^2 = \sum_n c_n^2 + \|\mathbf{e}\|^2 \Rightarrow$ each mode carries a well-defined share of the energy

The last point is the seed of the POD **energy spectrum** in Section 4.

Building an orthonormal basis: Gram–Schmidt

Given any independent $\{\mathbf{w}_1, \dots, \mathbf{w}_{n_b}\}$:

$$\tilde{\phi}_n = \mathbf{w}_n - \sum_{m < n} \langle \phi_m, \mathbf{w}_n \rangle \phi_m, \quad \phi_n = \tilde{\phi}_n / \|\tilde{\phi}_n\|$$

- subtract what is already representable, keep what is new
- same span, better conditioned basis (*this is the QR factorisation*)
- the SVD of Section 4 does something similar, and in addition chooses the *order* of the directions optimally instead of inheriting it

When a basis is a bad basis

Take $\text{span}\{1, x, \dots, x^{n_b-1}\}$ on $(0, 1)$ with the L^2 product:

$$A_{mn} = \int_0^1 x^{m+n-2} dx = \frac{1}{m+n-1} \quad (\text{the Hilbert matrix})$$

n_b	4	6	8	10
$\text{cond}(\mathbf{A})$	$1.6 \cdot 10^4$	$1.5 \cdot 10^7$	$1.5 \cdot 10^{10}$	$1.6 \cdot 10^{13}$

Over to you

The monomials and the Legendre polynomials up to degree $n_b - 1$ span the same space. Which gives the smaller approximation error, and which the smaller condition number?

The space is unchanged; the basis is the problem, and an orthogonal basis for the same space removes it.

Functions are vectors too

	vectors, $\mathbb{R}^{n_x}$	functions on Ω
object	$\mathbf{u} = (u_1, \dots, u_{n_x})^\top$	$u(x)$
addition, scaling	componentwise	pointwise
inner product	$\langle \mathbf{u}, \mathbf{v} \rangle = \sum_j u_j v_j$	$\langle u, v \rangle_\Omega = \int_\Omega u v \, d\Omega$
weighted	$\langle \mathbf{u}, \mathbf{v} \rangle_{\mathbf{W}} = \mathbf{u}^\top \mathbf{W} \mathbf{v}$	$\langle u, v \rangle_{w, \Omega} = \int_\Omega u v w \, d\Omega$
norm	$\ \mathbf{u}\ = \sqrt{\langle \mathbf{u}, \mathbf{u} \rangle}$	$\ u\ _{L^2} = \sqrt{\langle u, u \rangle_\Omega}$
approximation	$\tilde{\mathbf{u}} = \sum_n c_n \phi_n$	$u_{n_b} = \sum_n c_n \phi_n$
error	$\mathbf{e} = \mathbf{u} - \tilde{\mathbf{u}}$	$e(x) = u(x) - u_{n_b}(x)$
dimension	n_x	∞

Both are **Hilbert spaces**: a norm, an inner product, and a notion of orthogonality.

($L^2(\Omega)$ is used here as a working tool; completeness and measure theory play no role in what follows.)

One expansion, four readings of it

$$u(x) \approx u_{n_b}(x) = \sum_{n=1}^{n_b} c_n \phi_n(x)$$

reading	basis ϕ_n	where samples come from
interpolation / spectral	Fourier, Chebyshev	we choose them (collocation)
regression / statistics	RBFs, polynomials	given, noisy, scattered
signal processing	Fourier, wavelets	given, uniform
dimensionality reduction	learned from data	given, and n_b is what matters

Same equation every time. What changes is who chooses ϕ_n , and what we are allowed to sample.

Projection in function space

Define the error function, exactly as in $\mathbb{R}^{n_x}$:

$$e(x) = u(x) - u_{n_b}(x) = u(x) - \sum_{n=1}^{n_b} c_n \phi_n(x)$$

and impose that it be orthogonal to every basis function:

$$\langle \phi_m, e \rangle_{\Omega} = 0 \quad \Longleftrightarrow \quad \sum_n \underbrace{\left(\int_{\Omega} \phi_m \phi_n \, d\Omega \right)}_{A_{mn}} c_n = \underbrace{\int_{\Omega} \phi_m u \, d\Omega}_{b_m}$$

For an orthogonal basis this collapses to $c_n = \langle u, \phi_n \rangle_{\Omega} / \|\phi_n\|^2$, or $c_n = \langle u, \phi_n \rangle_{w, \Omega} / \|\phi_n\|_{w, \Omega}^2$ with a weight.

Over to you

Where did this derivation use that Ω is an interval, or that u depends on one variable?

The same condition, on both sides

	vectors	functions
error	$\mathbf{e} = \mathbf{u} - \sum_n c_n \phi_n$	$e(x) = u(x) - \sum_n c_n \phi_n(x)$
condition	$\langle \phi_m, \mathbf{e} \rangle = 0$	$\langle \phi_m, e \rangle_\Omega = 0$
written out	$\sum_j w_j \phi_m(x_j) e(x_j) = 0$	$\int_\Omega \phi_m(x) e(x) d\Omega = 0$
system	$\mathbf{A}\mathbf{c} = \mathbf{b}$	$\mathbf{A}\mathbf{c} = \mathbf{b}$

The two middle rows differ only by **how the integral is evaluated**: a weighted sum over samples, or an exact integral. The unknowns c_n , the matrix $\mathbf{A}$ and the vector $\mathbf{b}$ mean the same on both sides.

The discrete problem is not an approximation *of* the continuous one. It is the same problem, with the integral replaced by a quadrature.

Classical orthogonal families

family	domain	weight $w(x)$	natural for
Fourier e^{ikx}	$(0, 2\pi)$	1	periodic
$\sin(n\pi x)$	$(0, 1)$	1	homogeneous Dirichlet
$\cos(n\pi x)$	$(0, 1)$	1	homogeneous Neumann
Legendre P_n	$(-1, 1)$	1	general, unweighted
Chebyshev T_n	$(-1, 1)$	$(1 - x^2)^{-1/2}$	general, fast transforms

- each is orthogonal **with respect to its own weight**: for Chebyshev, $\int_{-1}^1 T_m T_n (1 - x^2)^{-1/2} dx = 0$ when $m \neq n$, while $\int_{-1}^1 T_m T_n dx \neq 0$
- T_n obeys $T_0 = 1$, $T_1 = x$, $T_{n+1} = 2xT_n - T_{n-1}$, and $T_n(\cos \theta) = \cos(n\theta)$, so Chebyshev is Fourier in disguise, on a stretched coordinate

Where these families come from: Sturm–Liouville

All of them solve the same kind of eigenvalue problem on $\Omega = (a, b)$:

$$-\frac{d}{dx} \left(p(x) \frac{d\phi_n}{dx} \right) + q(x) \phi_n = \lambda_n w(x) \phi_n, \quad \text{plus boundary conditions.}$$

- the operator on the left is **symmetric** in the w -weighted inner product, provided the boundary terms cancel
- symmetric operators have real eigenvalues and **orthogonal** eigenfunctions, exactly as symmetric matrices do; the weight w is the same in both places

family	p	q	w
$\sin(n\pi x)$ on $(0, 1)$	1	0	1
Legendre	$1 - x^2$	0	1
Chebyshev	$\sqrt{1 - x^2}$	0	$(1 - x^2)^{-1/2}$

Projecting an operator onto its own eigenfunctions is a diagonalisation. Section 3 lives on this.

Grids, collocation points, quadrature points

	what it is	what it is for
grid	points <i>plus</i> a connectivity: which point neighbours which, and what cell each one owns	finite differences and finite elements, which need neighbours to build stencils and elements
collocation points	a set of locations where an equation is enforced exactly	interpolation, collocation, pseudo-spectral methods
quadrature points	locations <i>with weights</i> w_j , chosen so that $\sum_j w_j g(x_j) \approx \int_{\Omega} g \, d\Omega$	evaluating inner products, hence $\mathbf{A}$ and $\mathbf{b}$

The same locations often serve all three roles, which is why the distinction gets lost. Projection needs only the third: **no connectivity is required**, which is what makes the machinery work on scattered data too.

Discrete inner products: from integrals to matrices

A quadrature rule replaces the integral by a weighted sum:

$$\langle u, v \rangle_{\Omega} = \int_{\Omega} u v \, d\Omega \approx \sum_{j=1}^{n_x} w_j u(x_j) v(x_j) = \mathbf{v}^{\top} \mathbf{W}_s \mathbf{u}, \quad \mathbf{W}_s = \text{diag}(w_1, \dots, w_{n_x}).$$

- w_j is the **measure** carried by point j : the length, area or volume of the piece of Ω it represents. On a uniform 1D grid $w_j = \Delta x$; in 2D it is a cell area
- $|\Omega| = \sum_j w_j$ is the size of the whole domain, so dividing by it turns the sum into an *average* rather than an integral. Keep the convention you chose and state it once
- choosing all $w_j = 1$ is a choice too: it says every sample counts equally, which is right for scattered data and wrong for a stretched grid

With $\mathbf{u}_*$ the vector of samples of u and $\Phi(\mathbf{x}_*) \in \mathbb{R}^{n_x \times n_b}$ the sampled basis,

$$\underbrace{\Phi^{\top} \mathbf{W}_s \Phi}_A \mathbf{c} = \underbrace{\Phi^{\top} \mathbf{W}_s \mathbf{u}_*}_b$$

Two ways for $\mathbf{A}$ to fail to be diagonal

cause	what to do about it
the basis is not orthogonal	nothing to fix: it is a modelling choice, and Cholesky solves the system
the basis <i>is</i> orthogonal but the quadrature cannot see it	refine or change the quadrature: this is a numerical accident, not a property of the basis

The second case is the common one. The products $\phi_m \phi_n$ oscillate roughly **twice as fast** as the basis functions, so a sampling that resolves the basis need not resolve $\mathbf{A}$.

- Fourier on a uniform grid: discretely orthogonal, exactly
- Chebyshev on a *uniform* grid: not discretely orthogonal at all
- Chebyshev at its own Gauss or Gauss–Lobatto points: discretely orthogonal, with the matching weights

(*Tutorial 1 makes all three visible on the same picture.*)

Convergence depends on smoothness

For an orthogonal expansion, Parseval turns the error into a tail sum:

$$\|u\|_{L^2}^2 = \sum_{n \geq 1} c_n^2 \quad \implies \quad \|u - u_{n_b}\|_{L^2}^2 = \sum_{n > n_b} c_n^2.$$

So **everything** is decided by how fast c_n decays.

regularity of u	coefficient decay	error $\ u - u_{n_b}\ _{L^2}$
analytic	$ c_n \sim e^{-\alpha n}$	exponential in n_b
C^p , p finite	$ c_n \sim n^{-(p+1)}$	algebraic, $n_b^{-p-1/2}$
discontinuous	$ c_n \sim n^{-1}$	very slow, Gibbs oscillations

Over to you

How would you measure the decay rate of c_n in an experiment, without knowing the exact solution?

The smoothness that matters is not always obvious

A sine series on $(0, 1)$ does not reconstruct u . It reconstructs the **odd periodic extension** of u , and it is *that* function whose smoothness sets the rate.

Take $u(x) = x(1 - x)$, a polynomial:

$$c_n = \frac{4\sqrt{2}}{n^3\pi^3} \quad (n \text{ odd}), \quad c_n = 0 \quad (n \text{ even})$$

so $|c_n| \sim n^{-3}$ and the error falls like $n_b^{-5/2}$,
not exponentially.

- $u(0) = u(1) = 0$, so the extension is continuous
- u' matches too
- but $u''(0^+) = -2$ against 0 from the other side: the second derivative jumps

Ask not "is u smooth?" but **"is the function the basis actually builds smooth?"** Compatibility with the boundary conditions is part of smoothness.

Which error can you reduce, and with which knob?

Three different things are usually called "the error". They respond to different actions.

error		set by	reduced by
truncation $\ u - Pu\ _{L^2}$, tail $\sum_{n>n_b} c_n^2$	the	the space : which n_b functions	more modes, or a basis better matched to u (<i>this is what POD automates</i>)
quadrature error in $\mathbf{A}$ and $\mathbf{b}$		the sampling	more or better placed points; it does <i>not</i> improve with n_b
conditioning amplification of round-off in $\mathbf{c}$		the basis , through $\text{cond}(\mathbf{A})$	orthogonalise, or change basis for the same space

Adding modes attacks only the first. Past a certain n_b it makes the third worse, and the total error starts to grow again.

(Tutorial 1 locates that crossover for the monomial basis.)

Projection is not interpolation

	interpolation	projection
condition	$u_{n_b}(x_j) = u(x_j)$ at n_b points	$\langle \phi_m, e \rangle_\Omega = 0$
uses	n_b point values	n_b integrals over Ω
error between points	unconstrained	accounted for by the integral
optimal in	nothing, in general	the chosen norm
sensitive to	node placement (<i>Runge</i>)	quadrature accuracy
noisy data	follows the noise exactly	averages it out
leads to	collocation, finite differences	Galerkin, FEM, spectral, POD

- they coincide in one special case: an orthogonal basis sampled at its own quadrature points, with the matching weights
- that coincidence is the whole design of pseudo-spectral methods

(Section 3 shows that both are members of the same weighted-residual family, differing only in the choice of test function.)

Two errors, and only one is our fault

$$\underbrace{u - u_h}_{\text{total}} = \underbrace{(u - Pu)}_{\text{best possible in the space}} + \underbrace{(Pu - u_h)}_{\text{what our method adds}}$$

- first term: the space is not rich enough, so change the space
- second term: the method is not doing its best, so fix the scheme

Always ask which of the two dominates.
We will use exactly this split to diagnose ROMs in Section 4.

Where we stand

1. A finite-dimensional space is chosen.
2. Orthogonality of the error to that space gives the coefficients.
3. The Gram matrix $\mathbf{A}$ encodes the geometry of the basis.
4. The inner product defines what "best" means.

But so far u was known. In a PDE we do *not* know the solution. We only know an equation it satisfies, so $\langle \phi_m, u \rangle_\Omega$ is not computable.

Section 3: replace "orthogonality of the **error**"
by "orthogonality of the **residual**".

Tutorial 1: what and why

Make this section concrete. Build $\mathbf{A}$, look at it, solve $\mathbf{A}\mathbf{c} = \mathbf{b}$, and see how the basis and the sampling control accuracy and conditioning.

Approximate on $\Omega = [-1, 1]$ two targets,

$$u_{\text{smooth}}(x) = x + e^{-3x^2} \sin(2\pi x), \quad u_{\text{kink}}(x) = |x|,$$

in four settings, all with the same number n_b of basis functions. Everything you need is one line of linear algebra repeated four times. The interest is entirely in how differently it behaves.

Tutorial 1: the four settings

	basis	weight $w(x)$	sampling	A diagonal?
1	monomials x^{n-1}	1	uniform grid	no
2	Fourier, $\cos(n\pi x), \sin(n\pi x)$	1	uniform grid	yes
3	Chebyshev T_n	$(1 - x^2)^{-1/2}$	uniform grid	?
4	Chebyshev T_n	Gauss–Lobatto weights	CGL points	?

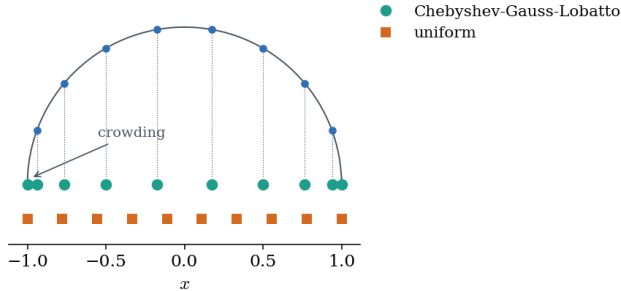
The **Chebyshev–Gauss–Lobatto** points are the extrema of T_{n_b} together with the two endpoints,

$$x_k = \cos\left(\frac{k\pi}{n_b}\right), \quad k = 0, 1, \dots, n_b,$$

clustered near $x = \pm 1$. Settings 3 and 4 use the **same basis** and differ only in where it is sampled. (*Answer the two question marks yourself before running the code.*)

Where the CGL points sit

$x_k = \cos(k\pi/N)$: equispaced angles, projected down



Equispaced *angles* on the half circle, projected onto the axis. The crowding towards $x = \pm 1$ is what removes the Runge phenomenon and makes the discrete orthogonality exact.

Tutorial 1: tasks

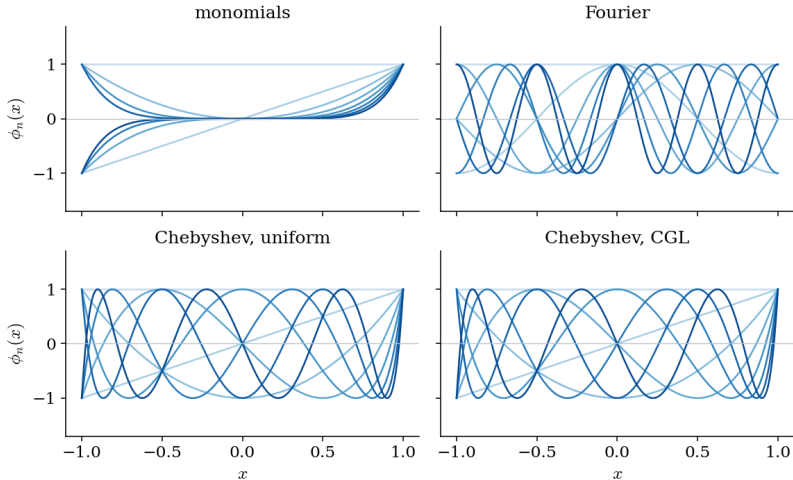
1. **Look at the bases.** Plot the first 8 elements of each of the four families. [\[produce plot\]](#)
2. **Test orthogonality, twice.** For each family compute

$$A_{mn}^{\text{cont}} = \int_{-1}^1 \phi_m \phi_n w \, dx \quad \text{and} \quad A_{mn}^{\text{disc}} = \sum_j w_j \phi_m(x_j) \phi_n(x_j)$$

and compare them. [\[implement\]](#)

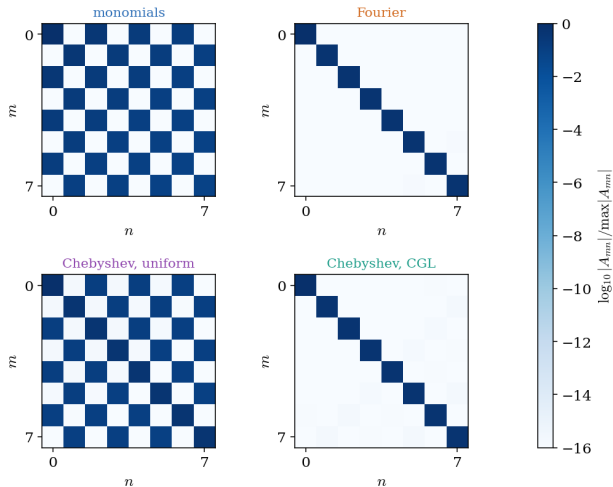
3. **Show $\mathbf{A}$.** Plot $|A_{mn}|$ as a contour or image for $n_b = 8$, for all four settings, on a common colour scale. [\[produce plot\]](#)
4. **Condition number.** Plot $\text{cond}(\mathbf{A})$ against n_b up to $n_b = 20$, semilog. [\[produce plot\]](#)
5. **Reconstruct.** Solve with Cholesky, plot u against u_{n_b} for $n_b = 2, 4, 8, 16$, and plot the error against n_b . [\[produce plot\]](#)

Tutorial 1: the bases



Monomials become nearly indistinguishable as n grows.

Tutorial 1: is the basis orthogonal?



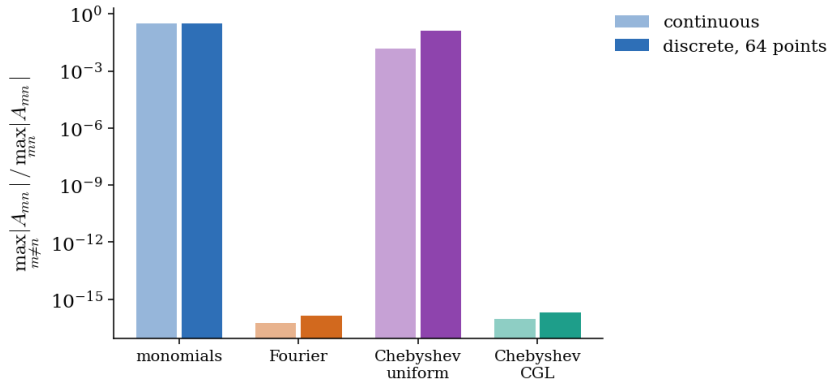
	continuous	discrete
monomials	$3 \cdot 10^{-1}$	$3 \cdot 10^{-1}$
Fourier	$6 \cdot 10^{-17}$	$1 \cdot 10^{-16}$
Cheb., uniform	$1 \cdot 10^{-2}$	$1 \cdot 10^{-1}$
Cheb., CGL	$9 \cdot 10^{-17}$	$2 \cdot 10^{-16}$

Both columns report the largest off-diagonal entry of $\mathbf{A}$ relative to the largest entry, so zero means an orthogonal basis. They differ only in how $A_{mn} = \langle \phi_m, \phi_n \rangle_{\Omega}$ is evaluated.

Continuous: the exact integral $\int \phi_m \phi_n \, dx$, a property of the functions alone.

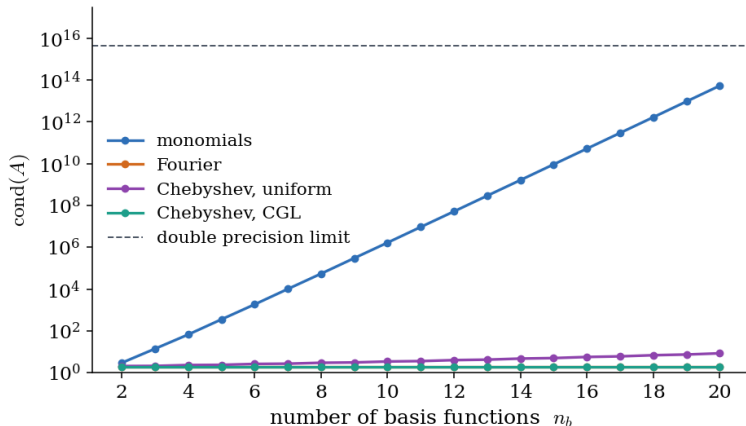
Discrete: the quadrature sum $\sum_k w_k \phi_m(x_k) \phi_n(x_k)$ over the points actually used, a property of the **pair** basis + points.

Tutorial 1: orthogonality, measured



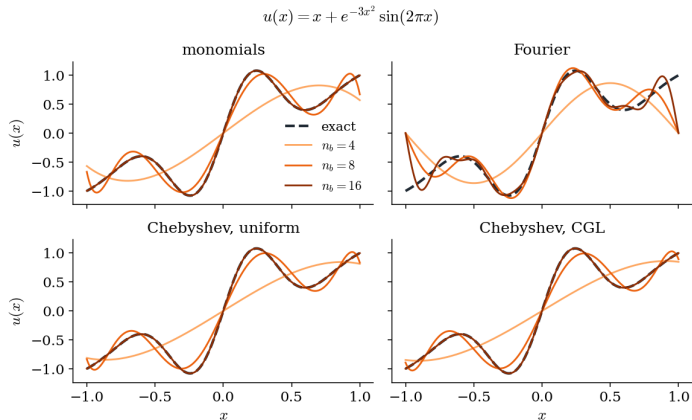
The largest off-diagonal entry of $\mathbf{A}$, relative to the largest entry. Continuous and discrete agree for Fourier and for CGL; for the uniform sampling of the same Chebyshev basis they do not.

Tutorial 1: conditioning



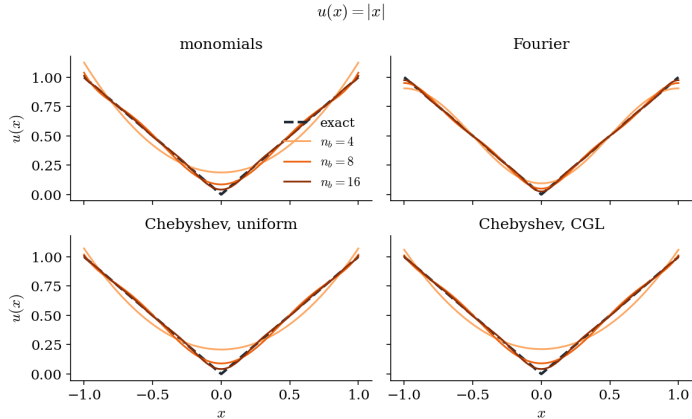
The monomial Gram matrix loses about one digit per added function and crosses the double precision limit near $n_b = 22$. The other three stay at $\text{cond}(\mathbf{A}) \approx 2$ forever.

Tutorial 1: a smooth target



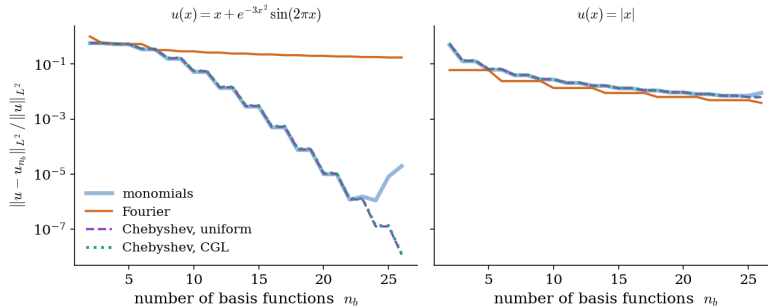
Fourier oscillates at the ends: the target is not periodic on $[-1, 1]$.

Tutorial 1: a target with a kink



One missing derivative slows every basis to the same algebraic rate. The regularity of the target sets the ceiling.

Tutorial 1: accuracy



Settings 1, 3 and 4 span the **same space**, so the curves coincide until round-off separates them near $n_b = 22$. Fourier stalls on the smooth target, which is **not periodic** on $[-1, 1]$; the kink limits every basis to algebraic convergence.

Tutorial 1: questions to answer with your plots

On orthogonality

Which families are orthogonal in the continuous sense? Which remain orthogonal once the integral becomes a sum? Settings 3 and 4 use the same basis: where exactly does the difference come from?

On conditioning and accuracy

At which n_b does the monomial system become useless in double precision? Does the *approximation* degrade at the same n_b , or later? Which of the two targets sets the achievable accuracy, and why?

Tutorial 1: what to conclude

1. $\mathbf{Ac} = \mathbf{b}$ is the whole of this section, in code.
2. Same **space**, different **basis**: the same best approximation, but wildly different conditioning.
3. Same **basis**, different **sampling**: discrete orthogonality is a property of the pair, not of the basis alone.
4. Same **everything**, different **target**: the decay rate belongs to the function, not to the method.

In Tutorial 3 the basis will be **chosen by the data**. This is the yardstick against which we will judge that choice.

Section 3

Galerkin projection of a PDE

-
- 3.1 The residual and the Galerkin condition
 - 3.2 Trial spaces, test spaces, boundary conditions
 - 3.3 Three answers to "which space?"
 - 3.4 Tutorial 2: one PDE, three discretisations

An abstract evolution PDE

$$\partial_t u = \mathcal{F}(u; \boldsymbol{\mu}), \quad x \in \Omega, \quad t > 0,$$

with boundary and initial conditions. Insert an ansatz that **separates space and time**:

$$u_{n_b}(x, t) = \sum_{n=1}^{n_b} a_n(t) \phi_n(x) \quad \Longleftrightarrow \quad u_{n_b}(x, t) = \boldsymbol{\phi}^\top(x) \boldsymbol{a}(t)$$

- $\phi_n(x)$: chosen once and fixed (*the approximation space*)
- $\boldsymbol{a}(t) \in \mathbb{R}^{n_b}$: the unknowns
- sampled on the grid, $\boldsymbol{u}_{n_b} = \boldsymbol{\Phi}(\boldsymbol{x}_*) \boldsymbol{a}$ with $\boldsymbol{\Phi} \in \mathbb{R}^{n_x \times n_b}$

(*Separation of variables with a prescribed spatial part. We are not solving an eigenvalue problem for ϕ_n .*)

The residual: the price of a finite space

$$\mathcal{R}(x, t) := \partial_t u_{n_b} - \mathcal{F}(u_{n_b}; \boldsymbol{\mu})$$

- in general $\mathcal{R} \not\equiv 0$: u_{n_b} cannot satisfy the PDE pointwise
- $\mathcal{R} \equiv 0$ would require the exact u to live in our space
- but $\mathcal{R}$ is **computable**, since it only involves u_{n_b}

n_b unknowns $\Rightarrow$ we may impose exactly n_b **scalar conditions** on $\mathcal{R}$.
Which ones? That is the entire design space.

Over to you

Propose one. Any rule that produces n_b scalar equations from $\mathcal{R}$ is admissible: what would you choose, and why?

The weighted-residual family

Pick n_b **test functions** ψ_m and impose $\langle \mathcal{R}, \psi_m \rangle_\Omega = 0$, $m = 1, \dots, n_b$.

method	ψ_m	meaning
collocation	$\delta(x - x_m)$	$\mathcal{R}(x_m) = 0$ at n_b points
finite volume	$\mathbf{1}_{\text{cell } m}$	cell-averaged conservation
least squares	$\partial \mathcal{R} / \partial a_m$	minimise $\ \mathcal{R}\ _{L^2}^2$
Galerkin	ϕ_m	residual $\perp$ trial space
Petrov–Galerkin	$\psi_m \neq \phi_m$	tailored weights

The two Petrov–Galerkin choices used later: **SUPG**, Streamline Upwind Petrov–Galerkin, and **LSPG**, Least-Squares Petrov–Galerkin.

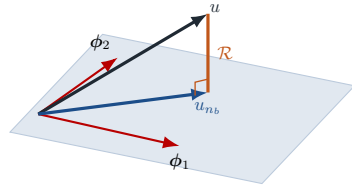
(All of these are legitimate. Galerkin is singled out because it inherits the best approximation geometry of Section 2 and preserves the symmetry and energy structure of the operator.)

Galerkin condition

$$\langle \mathcal{R}, \phi_m \rangle_{\Omega} = 0, \quad m = 1, \dots, n_b$$

- the residual is **invisible** to the approximation space
- whatever error remains cannot be corrected *within* the space
- the same picture as in Section 2, with the residual now playing the role of the error

(For a symmetric positive operator this makes u_{n_b} the best approximation of u in the energy norm, a result known as Cea's lemma.)



From PDE to ODEs *(step 1/2)*

Linear problem $\partial_t u = \mathcal{L}u + f$, ansatz $u_{n_b} = \sum_n a_n(t) \phi_n$:

$$\mathcal{R} = \sum_n \dot{a}_n \phi_n - \sum_n a_n \mathcal{L} \phi_n - f$$

Testing against ϕ_m and using linearity:

$$\sum_n \langle \phi_m, \phi_n \rangle_{\Omega} \dot{a}_n = \sum_n \langle \phi_m, \mathcal{L} \phi_n \rangle_{\Omega} a_n + \langle \phi_m, f \rangle_{\Omega}$$

(The time derivative came out of the integral because ϕ_n does not depend on t . This is the one place where a fixed basis is essential.)

From PDE to ODEs (step 2/2)

$$\mathbf{A} \dot{\mathbf{a}} = \mathbf{K} \mathbf{a} + \mathbf{f}, \quad \text{or, on the grid,} \quad \Phi^\top \mathbf{W}_s \Phi \dot{\mathbf{a}} = \dots$$

$$A_{mn} = \langle \phi_m, \phi_n \rangle_\Omega$$

Gram / mass matrix

$$K_{mn} = \langle \phi_m, \mathcal{L} \phi_n \rangle_\Omega$$

stiffness / operator matrix

$$f_m = \langle \phi_m, f \rangle_\Omega$$

load vector

A PDE in an infinite-dimensional space $\longrightarrow n_b$ coupled ODEs for $\mathbf{a}(t)$.

Over to you

What must be true of ϕ_n for $\mathbf{K}$ to be diagonal as well?

Nonlinear terms and the quadratic tensor

Projecting a quadratic term produces **one more index** than a linear one:

$$\dot{\mathbf{a}} = \mathbf{c}_0 + \mathbf{L}\mathbf{a} + \mathbf{C} : (\mathbf{a} \otimes \mathbf{a}), \quad [\mathbf{C} : (\mathbf{a} \otimes \mathbf{a})]_m = \sum_{n,k} C_{mnk} a_n a_k$$

- Burgers and Navier–Stokes both give a three-index tensor
- assembling $\mathbf{C}$ costs $\mathcal{O}(n_b^3)$, **but only once, offline**
- in code, `np.einsum('mnk,n,k->m', C, s, s)`
- a **general** nonlinearity has no such closed form, and its cost stays tied to the fine grid. We return to this in Section 4.

(This same structure returns in Section 4 as the POD-Galerkin model.)

Building $\mathcal{C}$ by hand

Take the convective term $-v \partial_x v$ with $n_b = 3$ and $v = a_1 \phi_1 + a_2 \phi_2 + a_3 \phi_3$. Then

$$v \partial_x v = \left(\sum_n a_n \phi_n \right) \left(\sum_k a_k \phi'_k \right) = \sum_{n=1}^3 \sum_{k=1}^3 a_n a_k \phi_n \phi'_k$$

Testing with ϕ_m and moving the coefficients out of the integral,

$$\begin{aligned} \langle v \partial_x v, \phi_m \rangle_\Omega &= \sum_{n,k} a_n a_k \underbrace{\langle \phi_n \phi'_k, \phi_m \rangle_\Omega}_{-C_{mnk}} \\ \Rightarrow C_{mnk} &= - \int_\Omega \phi_m(x) \phi_n(x) \phi'_k(x) dx \end{aligned}$$

(m labels the equation, n and k label the two factors of the product. Twenty-seven integrals here, n_b^3 in general.)

Building $\mathbf{C}$ by hand: the bookkeeping

For $n_b = 3$ the first equation ($m = 1$) reads

$$\begin{aligned}
 [\mathbf{C} : (\mathbf{a} \otimes \mathbf{a})]_1 &= \sum_{n,k=1}^3 C_{1nk} a_n a_k = & C_{111} a_1^2 &+ C_{112} a_1 a_2 &+ C_{113} a_1 a_3 \\
 & & + C_{121} a_2 a_1 &+ C_{122} a_2^2 &+ C_{123} a_2 a_3 \\
 & & + C_{131} a_3 a_1 &+ C_{132} a_3 a_2 &+ C_{133} a_3^2
 \end{aligned}$$

- $\mathbf{C}$ is **not** symmetric in n and k : the second index carries ϕ_n , the third carries the derivative ϕ'_k
- only the symmetric part of $a_n a_k$ survives, so C_{mnk} and C_{mkn} always act together
- all n_b^3 integrals are computed once, before time stepping starts

What does one right-hand side evaluation cost, and how does that compare with one step of the finite-difference solver?

Trial and test spaces

- **Trial space** $\mathcal{V}_N = \text{span}\{\phi_n\}$: where the solution is sought (*it must satisfy the essential boundary conditions*)
- **Test space** $\mathcal{W} = \text{span}\{\psi_m\}$: what the residual is tested against (*the homogeneous counterparts of those conditions*)

$$\text{find } u_{n_b} \in \mathcal{V}_N \quad \text{such that} \quad \langle \mathcal{R}(u_{n_b}), \psi_m \rangle_{\Omega} = 0 \quad \forall \psi_m \in \mathcal{W}$$

Galerkin	$\mathcal{W} = \mathcal{V}_N$, the same functions on both sides
Petrov–Galerkin	$\mathcal{W} \neq \mathcal{V}_N$, chosen for a purpose

Both need $\dim \mathcal{W} = \dim \mathcal{V}_N$, otherwise the system is over or under determined.

Why the Galerkin choice is special

With $\mathcal{W} = \mathcal{V}_N$, the projected operator inherits the structure of the continuous one:

$$K_{mn} = \langle \phi_m, \mathcal{L}\phi_n \rangle_{\Omega} \quad \implies \quad \mathcal{L} = \mathcal{L}^* \Rightarrow \mathbf{K} = \mathbf{K}^{\top}$$

- **symmetry is preserved:** a self-adjoint operator stays self-adjoint after projection, and a skew-symmetric convective term stays skew-symmetric
- **energy is preserved:** u_{n_b} is itself an admissible test function, so the discrete energy balance mirrors the continuous one
- for a coercive symmetric problem this gives **optimality**: u_{n_b} is the closest element of $\mathcal{V}_N$ to u in the energy norm

Galerkin is the choice that lets the discrete problem inherit the physics of the continuous one.

Petrov–Galerkin: choosing a different test space

The residual condition $\langle \mathcal{R}, \psi_m \rangle_\Omega = 0$ does not require $\psi_m = \phi_m$. Changing the test space changes *which* n_b pieces of information we extract from the residual.

method	test functions ψ_m	what it buys
Galerkin	ϕ_m	symmetry, energy, optimality
SUPG	$\phi_m + \tau \mathbf{a} \cdot \nabla \phi_m$	stability in convection-dominated flow
least-squares	$\mathcal{L}\phi_m$	a symmetric positive system for any $\mathcal{L}$
LSPG	$\mathbf{J} \Phi$, with $\mathbf{J}$ the Jacobian	minimises the residual of the <i>discrete</i> problem

Over to you

With $\psi_m \neq \phi_m$, what happens to the symmetry of $K_{mn} = \langle \psi_m, \mathcal{L}\phi_n \rangle_\Omega$?

What each choice minimises

Each method is the stationarity condition of a different quantity.

method	minimises	requires
Galerkin	$\ u - u_{n_b}\ _a$, the energy norm of the error	$\mathcal{L}$ symmetric positive
least-squares	$\ \mathcal{R}\ _{L^2}$, the residual itself	nothing
LSPG	$\ \mathbf{r}(\mathbf{a})\ _2$, the <i>discrete</i> residual	a chosen discretisation

- minimising the residual is not the same as minimising the error, and the gap between them is controlled by the conditioning of $\mathcal{L}$
- least-squares always gives a symmetric positive system, at the price of squaring that conditioning
- LSPG is the version used for reduced models, where the discrete residual is the only one available

Watching Galerkin misbehave: the smallest example

Steady convection and diffusion on $(0, 1)$, with $a > 0$ constant:

$$a \partial_x u = \nu \partial_{xx} u, \quad u(0) = 0, \quad u(1) = 1.$$

The exact solution is a boundary layer of thickness ν/a at $x = 1$:

$$u(x) = \frac{e^{ax/\nu} - 1}{e^{a/\nu} - 1}.$$

Galerkin with piecewise linear elements on a uniform mesh gives, at an interior node,

$$\frac{a}{2\Delta x} (a_{j+1} - a_{j-1}) = \frac{\nu}{\Delta x^2} (a_{j+1} - 2a_j + a_{j-1}).$$

- this is exactly the **central difference** scheme
- the convective term produced a **skew-symmetric** matrix, so it carries no dissipation at all

(Write both matrices side by side: the diffusion part is symmetric and negative definite, the convection part is skew-symmetric.)

Where the oscillations come from

Look for solutions of the form $a_j = \kappa^j$ and define the **cell Péclet number** $Pe = a\Delta x/(2\nu)$. The recurrence gives

$$\kappa = \frac{1 + Pe}{1 - Pe}.$$

- $Pe < 1$: $\kappa > 0$, the solution grows monotonically, as it should
- $Pe = 1$: $\kappa \rightarrow \infty$, the scheme breaks down
- $Pe > 1$: $\kappa < 0$, so a_j **alternates in sign** from node to node

The oscillation is not round-off. It is the exact solution of the discrete problem.

Galerkin is optimal in the energy norm of a *symmetric* problem. Convection is not symmetric, so the guarantee does not apply.

Worth checking

With $a = 1$ and $\nu = 10^{-2}$, how fine must the mesh be before $Pe < 1$? Compare that with the thickness of the layer you are trying to resolve.

Two cures, and what each one costs

cure	how	price
refine the mesh	make $\Delta x < 2\nu/a$ everywhere the layer sits	cost grows as the layer thins; hopeless at high Reynolds number
change the test space	$\psi_m = \phi_m + \tau a \partial_x \phi_m$, biased upwind	no longer Galerkin, so the optimality of the previous slides is lost

- with the optimal τ the second choice reproduces the exact nodal values of this problem; in general it adds a controlled amount of dissipation, aligned with the flow
- this is the origin of **SUPG**, and the same tension returns for reduced models

A symmetric test space cannot see the direction of transport.

Essential and natural boundary conditions

	essential (Dirichlet)	natural (Neumann)
prescribes	the solution itself	a flux or a derivative
imposed on	the trial space	the weak form
mechanism	the basis must satisfy it	a boundary term
test functions	vanish there	are unconstrained there
if you forget	you solve in the wrong space	you solve the wrong equation

A natural condition is satisfied *approximately*, in the limit of refinement, while an essential one is satisfied *exactly* by construction. This asymmetry is why the next slide matters.

Lifting: making the boundary conditions homogeneous

A set of functions with $u|_{\partial\Omega} = u_{bc} \neq 0$ is an **affine set**, not a subspace: no basis can span it.

$$u(x, t) = \underbrace{h(x)}_{\text{known, carries the BC data}} + \underbrace{v(x, t)}_{\text{homogeneous BCs}}$$

- now every ϕ_n can satisfy the *homogeneous* conditions
- h generates extra constant and linear terms in the projected system
- any admissible h works, and a smoother h gives a better behaved v
- a time-dependent boundary datum makes $h = h(x, t)$ and adds a $-\partial_t h$ source (*we keep h steady today*)

(Section 4 uses the same algebraic device under the name of mean subtraction.)

Weak form: projection first, integration by parts second

The general template of this section, $\mathcal{L}$ linear and $\mathbf{N}$ quadratic:

$$\mathcal{R} = \partial_t u_{n_b} - c_0 - \mathcal{L}u_{n_b} - \mathbf{N}(u_{n_b})$$

1. Project. The Galerkin condition, unchanged, with $\psi_m = \phi_m$: $\langle \mathcal{R}, \phi_m \rangle_\Omega = 0$.

2. Integrate by parts, only on the terms carrying two derivatives:

$$\int_{\Omega} \phi_m \nu \partial_{xx} u_{n_b} \, dx = \underbrace{[\nu \phi_m \partial_x u_{n_b}]_{\partial\Omega}}_{\text{natural conditions live here}} - \nu \int_{\Omega} \partial_x \phi_m \partial_x u_{n_b} \, dx$$

- first-order and nonlinear terms are left alone: they already ask for only one derivative
- what is gained is admissibility, not accuracy: for smooth u the weak and strong statements are equivalent

Weak form = Galerkin condition + integration by parts where the space cannot afford the derivative.

Finite differences: approximate the operator

$$\partial_x u(x_i) \approx \frac{u_{i+1} - u_{i-1}}{2\Delta x}, \quad \partial_{xx} u(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2}$$

- the unknowns are **point values**, not coefficients
- everything follows from Taylor expansions and truncation error
- no trial space, no test space, no inner product

A different philosophy: **discretise the differential operator** rather than the function.

(In fairness, finite differences can be reinterpreted as collocation on a local polynomial trial space.)

Spectral methods: a few global functions

$$v_{n_b}(x, t) = \sum_{n=1}^{n_b} a_n(t) \phi_n(x), \quad \phi_n \text{ smooth, supported on the whole domain}$$

- the basis is chosen so that each ϕ_n *already* satisfies the homogeneous boundary conditions (*hence the lifting*)
- exponential convergence for smooth solutions (*Section 2*)
- the price is global coupling, dense operators and rigid geometry

Checked in Tutorial 2

How many sine modes are needed to match a finite-difference grid of 200 points on our problem, and what happens to that count if the solution develops a steep front?

Spectral Galerkin: the operators collapse

With sines scaled so that $\Phi^\top \Phi = \mathbf{I}$:

$$A_{mn} = \delta_{mn}, \quad \phi_n''(x) = \lambda_n \phi_n(x), \quad \Phi_{xx}(\mathbf{x}_*) = \mathbf{\Lambda} \Phi(\mathbf{x}_*)$$

- $\mathbf{A} = \mathbf{I}$, so there is no linear system for the time derivative
- diffusion is **diagonal**, so the linear dynamics decouples mode by mode
- this happens because the ϕ_n are **eigenfunctions** of ∂_{xx} with these boundary conditions

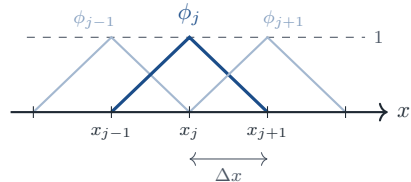
Choosing eigenfunctions of the operator diagonalises the linear part.
The nonlinear term never diagonalises, and that is where the work is.

Finite elements: a flexible *local* space

- partition Ω into elements
- low-order polynomials on each
- glue into globally continuous functions
- ϕ_n : **compact support**, $\phi_n(x_j) = \delta_{nj}$

$$u_h(x, t) = \sum_n a_n(t) \phi_n(x), \quad a_n = u_h(x_n, t)$$

(the coefficients are nodal values)



each ϕ_j is nonzero on two elements only

The element matrices

On one element $[x_j, x_{j+1}]$ of length Δx only two of the ϕ_n are nonzero. Call their restrictions ϕ_1^e, ϕ_2^e and write them in the local coordinate $\xi = (x - x_j)/\Delta x \in [0, 1]$:

$$\phi_1^e(\xi) = 1 - \xi, \quad \phi_2^e(\xi) = \xi, \quad (\phi_1^e)' = -\frac{1}{\Delta x}, \quad (\phi_2^e)' = +\frac{1}{\Delta x}.$$

Two of the blocks that element contributes:

$$A_{pq}^e = \int_{x_j}^{x_{j+1}} \phi_p^e \phi_q^e dx = \frac{\Delta x}{6} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad K_{pq}^e = \int_{x_j}^{x_{j+1}} (\phi_p^e)' (\phi_q^e)' dx = \frac{1}{\Delta x} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

Over to you

Compute $A_{11}^e = \int_0^1 (1 - \xi)^2 \Delta x d\xi$ and check the factor $\Delta x/3$. Why is K^e singular?

The two blocks that are not symmetric

The lifting and the nonlinearity contribute in exactly the same way, one element at a time. With h_e the value of h on the element,

$$B_{pq}^e = \int \phi_p^e h (\phi_q^e)' dx = \frac{h_e}{2} \begin{bmatrix} -1 & +1 \\ -1 & +1 \end{bmatrix}, \quad C_{pqr}^e = - \int \phi_p^e \phi_q^e (\phi_r^e)' dx = \mp \frac{1}{6} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

the sign of C^e following that of $(\phi_r^e)'$.

- B^e is **not symmetric**: convection has a direction, and a symmetric operator cannot carry one
- Δx cancels in C^e , so the quadratic term does not stiffen with the mesh, unlike K^e

(Same three loops as for A^e and K^e , one index deeper for C^e .)

Assembly, and the system you will actually build

Each element adds its blocks on the diagonal, overlapping with its neighbours at the shared node. For three elements:

$$\mathbf{A} = \frac{\Delta x}{6} \begin{bmatrix} 2 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 4 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix}, \quad \mathbf{K} = \frac{1}{\Delta x} \begin{bmatrix} 1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{bmatrix}$$

Collecting every block, the projected system is the template of this section:

$$\mathbf{A} \dot{\mathbf{a}} = \underbrace{\mathbf{g} + \alpha \mathbf{f}}_{c_0} + \underbrace{(\mathbf{A} - \mathbf{B} - \nu \mathbf{K})}_{L} \mathbf{a} + \mathbf{C} : (\mathbf{a} \otimes \mathbf{a})$$

- the 4 on the diagonal of $\mathbf{A}$ is $2 + 2$: node 1 belongs to two elements
- the interior rows of $\mathbf{K}$ read $(-1, 2, -1)/\Delta x$, the finite difference Laplacian reached from a different starting point

Same machine, different space

$$\text{Spectral: } v_{n_b} = \sum_n a_n(t) \underbrace{\phi_n^{\text{sine}}(x)}_{\text{global, analytic}}$$

$$\text{FEM: } u_h = \sum_n a_n(t) \underbrace{\phi_n^{\text{hat}}(x)}_{\text{local, mesh-based}}$$

$$\text{POD: } v_{n_r} = \sum_r a_r(t) \underbrace{\phi_r^{\text{POD}}(x)}_{\text{global, learned}}$$

Identical projection step. Only the space changes.

What happens after projection: time integration

All three methods leave the same object:

$$\mathbf{A}\dot{\mathbf{a}} = \mathbf{r}(\mathbf{a}; \boldsymbol{\mu}), \quad \mathbf{a} \in \mathbb{R}^{n_b}$$

- **explicit** (Euler, RK4): cheap steps, but diffusion imposes $\Delta t \lesssim \Delta x^2/\nu$, which is **severe on fine grids**
- **semi-implicit**: implicit diffusion with explicit convection, the usual compromise
- with $\mathbf{A} \neq \mathbf{I}$, as in FEM, factorise once or lump the mass matrix
- with $\mathbf{A} = \mathbf{I}$, as in spectral or POD bases, the system is already explicit

A reduced model with $n_r \sim 10$ states removes both the size penalty and the stiffness penalty.

Explicit Runge–Kutta

For $\dot{\mathbf{a}} = \mathbf{r}(\mathbf{a}, t)$ the classical fourth-order scheme evaluates the right-hand side four times per step:

$$\begin{aligned}
 \mathbf{k}_1 &= \mathbf{r}(\mathbf{a}^n, t_n) & \mathbf{k}_2 &= \mathbf{r}\left(\mathbf{a}^n + \frac{\Delta t}{2} \mathbf{k}_1, t_n + \frac{\Delta t}{2}\right) \\
 \mathbf{k}_3 &= \mathbf{r}\left(\mathbf{a}^n + \frac{\Delta t}{2} \mathbf{k}_2, t_n + \frac{\Delta t}{2}\right) & \mathbf{k}_4 &= \mathbf{r}(\mathbf{a}^n + \Delta t \mathbf{k}_3, t_n + \Delta t) \\
 \mathbf{a}^{n+1} &= \mathbf{a}^n + \frac{\Delta t}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)
 \end{aligned}$$

- no linear system to solve, four right-hand side evaluations per step, error $\mathcal{O}(\Delta t^5)$
- the step is limited by **stability**, not accuracy: $\Delta t \lesssim \Delta x^2/\nu$ from diffusion, $\Delta t \lesssim \Delta x/|u|$ from convection

Over to you

On 200 points with $\nu = 0.03$, what does the diffusive limit give for Δt , and how many steps to reach $t = 1$?

Semi-implicit splitting

Split the right-hand side, then take the stiff part at $n + 1$ and the rest at n :

$$\dot{\mathbf{a}} = \underbrace{\mathbf{L}\mathbf{a}}_{\text{stiff, linear}} + \underbrace{\mathbf{n}(\mathbf{a})}_{\text{non-stiff, nonlinear}}$$

$$\frac{\mathbf{a}^{n+1} - \mathbf{a}^n}{\Delta t} = \mathbf{L}\mathbf{a}^{n+1} + \mathbf{n}(\mathbf{a}^n) \quad \Longrightarrow \quad (\mathbf{I} - \Delta t \mathbf{L})\mathbf{a}^{n+1} = \mathbf{a}^n + \Delta t \mathbf{n}(\mathbf{a}^n)$$

- the diffusive step limit disappears; only the convective one remains
- one linear solve per step, with a matrix that does not change: **factorise once**, reuse at every step
- $\mathbf{L}$ is small and dense for a spectral or POD basis, sparse for finite elements

Over to you

Why can the nonlinear term not simply be treated implicitly as well, and what would it cost if we did?

Comparison

	FD	Spectral	FEM	POD
starts from	operator	solution	solution	solution
basis	none	prescribed	prescribed	learned
support	stencil	global	local	global
A	none	I	sparse	I
convergence	Δx^2	exponential	Δx^p	data-dependent
geometry	structured	rigid	arbitrary	inherited
typical size	10^3 – 10^8	10^1 – 10^3	10^3 – 10^8	10^0 – 10^2
purpose	simple solver	high accuracy	flexible solver	model reduction

Spectral and FEM spaces are designed to solve the PDE *well*.
 POD builds a space adapted to the trajectories we expect.

The running problem *(used in Tutorials 2 and 3)*

$$\partial_t u + u \partial_x u = \nu \partial_{xx} u + \alpha f(x), \quad x \in (0, 1)$$

$$u(0, t) = 1, \quad u(1, t) = 0, \quad u(x, 0) = 1 - x$$

$$f(x) = \exp\left[-\frac{(x - x_f)^2}{2\sigma_f^2}\right], \quad x_f = 0.65, \sigma_f = 0.08$$

Parameters $\boldsymbol{\mu} = (\nu, \alpha)$ on $\nu \in [0.01, 0.10]$, $\alpha \in [0.5, 1.5]$. Tutorial 2 solves one point of it, $\boldsymbol{\mu} = (0.03, 1.0)$; Tutorial 3 covers the whole box. (*Forced viscous Burgers: nonlinear convection, diffusion, a localised source, one non-homogeneous and one homogeneous Dirichlet condition, two parameters. Even at the lowest ν the cell Péclet number stays below one on $n_x = 200$, so no shock capturing is needed.*)

Lifting, once for the whole day [derive in class]

$$u = h(x) + v(x, t), \quad h(x) = 1 - x \Rightarrow v(0, t) = v(1, t) = 0, \quad v(x, 0) = 0$$

With $h' = -1$, $h'' = 0$, grouping by structure:

$$\partial_t v = \underbrace{(1 - x) + \alpha f(x)}_{\text{constant} \rightarrow c_0} + \underbrace{\nu \partial_{xx} v + v - (1 - x) \partial_x v}_{\text{linear} \rightarrow L} - \underbrace{v \partial_x v}_{\text{quadratic} \rightarrow C}$$

- exactly the template of the previous slides, before any basis is chosen
- **affine** in ν and in α by construction
- $\mathbf{a}(0) = \mathbf{0}$, because the initial condition *is* the lifting

Tutorial 2: assignment

Solve the same problem three ways.

part	method	unknowns	you do
A	finite differences, $n_x = 200$	point values u_i	read and run
B	spectral Galerkin, sine basis	$\mathbf{a}(t)$	assemble $\mathbf{c}_0, \mathbf{L}, \mathbf{C}, \mathbf{f}$
C	linear finite elements	nodal $\mathbf{a}(t)$	read and run

Produce: the three solutions overlaid at $t = 0.5$ [\[produce plot\]](#); the error against the number of degrees of freedom [\[produce plot\]](#); the wall-clock time of one finite-difference run (*the baseline for Tutorial 3*). (Parts B and C both use the lifting above, so only the basis differs.)

Tutorial 2: the projection to derive **[derive in class]**

With $\Phi^\top \Phi = \mathbf{I}$ and $\phi_n'' = \lambda_n \phi_n$:

$$c_{0,m} = \langle \phi_m, 1 - x \rangle_\Omega + \alpha \langle \phi_m, f \rangle_\Omega$$

$$L_{mn} = \nu \lambda_n \delta_{mn} + \delta_{mn} - \langle \phi_m, (1 - x) \phi_n' \rangle_\Omega$$

$$C_{mnk} = - \langle \phi_m, \phi_n \phi_k' \rangle_\Omega$$

$$\Rightarrow \quad \dot{\mathbf{a}} = \mathbf{c}_0(\alpha) + \mathbf{L}(\nu) \mathbf{a} + \mathbf{C} : (\mathbf{a} \otimes \mathbf{a}), \quad \mathbf{a}(0) = \mathbf{0}$$

Watch out for: the normalisation of the basis; comparing v_{n_b} against u instead of $h + v_{n_b}$; an under-resolved quadrature for $\mathbf{C}$.

Tutorial 2: what the results should show

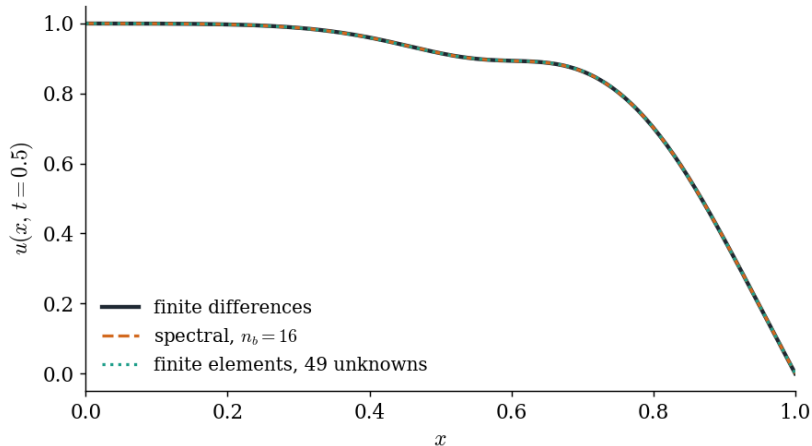
Questions to answer with your plots

1. How many sine modes are needed before the spectral solution becomes indistinguishable from the finite-difference reference?
2. On a plot of error against degrees of freedom, which curve is straight on a log-log axis and which is straight on a semilog axis? Why?
3. Which part of the three codes actually changed between B and C?

- three codes, **one derivation**
- and yet none of these bases knows anything about $f(x)$, ν or α

Could four **well chosen** functions do what sixteen sines do here?

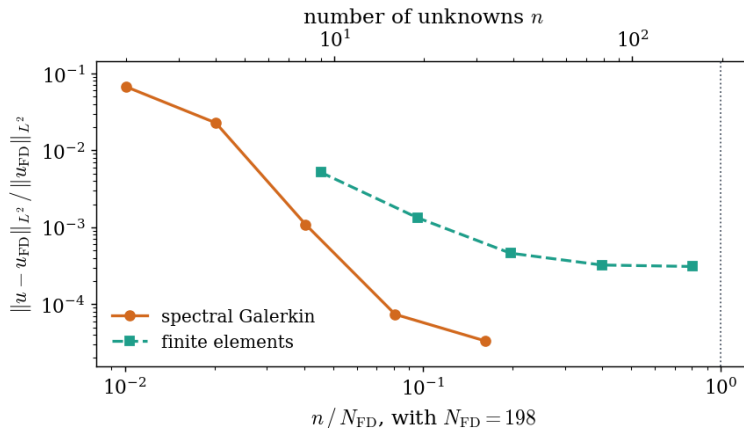
Tutorial 2: the three solutions at $t = 0.5$



Sixteen sine modes and forty-nine finite-element nodes sit on top of a two-hundred-point reference.

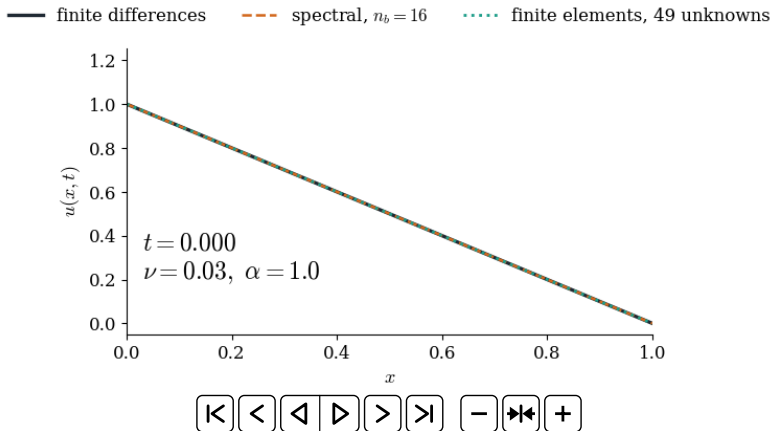
One derivation, three choices of Φ .

Tutorial 2: accuracy against the size of the model



The abscissa is n/N_{FD} , the model size relative to the reference. At 10^{-4} the spectral basis needs about 8% of it; the finite elements have already stalled, at the accuracy of the reference itself.

Tutorial 2: the three solutions in time



The source at $x_f = 0.65$ builds the bump, convection carries it right and diffusion smooths it. The three discretisations stay indistinguishable throughout.

Section 4

POD and POD-Galerkin reduced-order models

- 4.1 Letting the data choose the basis
- 4.2 The POD, from the continuous problem to the SVD
- 4.3 Practical choices
- 4.4 POD-Galerkin
- 4.5 Diagnostics and limitations
- 4.6 Tutorial 3: POD and a POD-Galerkin ROM

The question

The expensive model has **already** been run many times: design studies, calibration, parameter sweeps, past operation.

Can we compress these previously generated solutions into a small set of dominant spatial structures, and reuse them online?

Now the basis is **not prescribed**. This is the fourth reading of the expansion we met in Section 2: all the attention goes to n_r , not to the analytic form of ϕ_r . (*We usually have no formula for these modes, only their samples.*)

The continuous setting

Before any grid, the object we want to compress is a **random field**:

$$u(x, \xi), \quad x \in \Omega, \quad \xi = \text{the realisation (time instant, parameter, experiment)}$$

- ξ indexes the ensemble: for us, one ξ is one instant of one run
- the ensemble average is written $E[\cdot]$
- the space is $L^2(\Omega)$, with the inner product of Section 2

The question is not *which coefficients* in a given space, as in Section 2, but **which space**.

(Everything on the next two slides is independent of the discretisation. The grid enters only afterwards, through the quadrature.)

The optimisation problem behind POD [derive in class]

What we actually want is the **single** function ϕ that reconstructs the field best, which is a minimisation:

$$\min_{\phi, a} E \left[\|u - a(\xi) \phi\|_{L^2}^2 \right] \quad \text{subject to} \quad \int_{\Omega} \phi^2 \, d\Omega = 1$$

Minimising over a first gives $a = \langle u, \phi \rangle_{\Omega}$, and then **energy splits exactly**:

$$\underbrace{E \left[\|u\|_{L^2}^2 \right]}_{\text{fixed by the data}} = \underbrace{E \left[|\langle u, \phi \rangle_{\Omega}|^2 \right]}_{\text{captured}} + \underbrace{E \left[\|u - a\phi\|_{L^2}^2 \right]}_{\text{left over}}$$

Minimising what is left over = **maximising what is captured**,
 $\max_{\phi} E \left[|\langle u, \phi \rangle_{\Omega}|^2 \right]$, and only this is written from here on.

(The constraint is not cosmetic: without it, scaling ϕ changes the objective and there is no maximum.)

Reading the maximisation

$$\max_{\phi} E \left[\left| \langle u(x, \xi), \phi(x) \rangle_{\Omega} \right|^2 \right] \quad \text{subject to} \quad \int_{\Omega} \phi^2 \, d\Omega = 1$$

- "captures the most energy on average" is the definition of a **coherent structure**
- the word **energy** is exact here, not a metaphor: it is the L^2 norm of the field, split by the identity on the previous slide
- the next mode repeats the problem on what the first one left behind

Over to you

Expand the objective. Which two-point quantity appears once the expectation is taken inside the two integrals?

From the optimisation problem to an eigenproblem **[derive in class]**

Writing the objective with the covariance kernel $C(x, x') = E\{u(x, \xi) u(x', \xi)\}$,

$$E[|\langle u, \phi \rangle_{\Omega}|^2] = \int_{\Omega} \int_{\Omega} \phi(x) C(x, x') \phi(x') dx dx'$$

A constrained maximisation of a quadratic form is solved by a Lagrange multiplier, and stationarity gives the Fredholm eigenvalue problem

$$\int_{\Omega} C(x, x') \phi_i(x') dx' = \lambda_i \phi_i(x)$$

- C is symmetric and positive, so the ϕ_i are orthogonal and $\lambda_1 \geq \lambda_2 \geq \dots \geq 0$
- the multiplier is the value of the objective, so λ_i ranks the modes
- $E[a_i a_j] = \lambda_i \delta_{ij}$: the coefficients are uncorrelated

(A generalised Fourier series whose basis is dictated by the data: the Karhunen–Loève theorem.)

Now discretise: snapshots and the dataset matrix

The kernel C is never available in closed form. What we have is a finite ensemble, sampled on a grid:

$$\mathbf{D} = [\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_{n_t}] \in \mathbb{R}^{n_s \times n_t}, \quad \mathbf{d}_k = u(\mathbf{x}_i, \xi_k) \in \mathbb{R}^{n_s}$$

- each column is a snapshot, at a given time **and** a given parameter
- each row is the history of one spatial location
- for vector fields, stack the components inside each $\mathbf{d}_k$

Over to you

What would $\mathbf{D}$ look like for a purely steady solution?

(Sampling design matters. Too few instants and the transient is missed, too many nearly identical late-time states and the steady state dominates.)

From the kernel to a matrix

Two approximations, and only two:

continuous	discrete	what replaces what
$E[\cdot]$	$\frac{1}{n_t} \sum_k$	ensemble average by a finite sum
$\int_{\Omega} f \, d\Omega$	$\sum_i w_i f(\mathbf{x}_i)$	integral by a quadrature, weights in $\mathbf{W}_s$

The kernel becomes the **covariance matrix**, and the Fredholm integral becomes a matrix product:

$$C(x, x') \longrightarrow \mathbf{C} = \frac{1}{n_t} \mathbf{D} \mathbf{D}^T \in \mathbb{R}^{n_s \times n_s}, \quad \int_{\Omega} C(x, x') \phi \, dx' \longrightarrow \mathbf{C} \mathbf{W}_s \phi$$

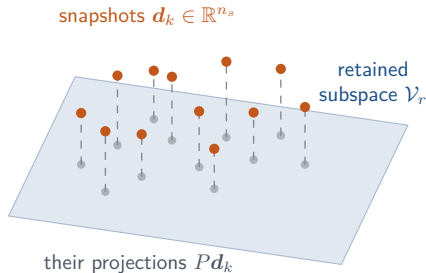
$\mathbf{C} \mathbf{W}_s \phi_r = \lambda_r \phi_r$ is the Fredholm problem, evaluated with a quadrature rule.

The observation that makes it work

- snapshots are highly **redundant**, because physics constrains them
- a random vector of $\mathbb{R}^{n_s}$ looks nothing like a solution
- the effective dimension is set by the dynamics, not by n_s

So the question is not whether a small subspace exists, but how to find the best one.

POD chooses the subspace that makes all the dashed segments as short as possible.



Two eigenproblems, one spectrum

With the discrete inner products of Section 2, $\langle \mathbf{a}, \mathbf{b} \rangle_{\Omega} = \mathbf{b}^{\top} \mathbf{W}_s \mathbf{a}$ and $\langle \mathbf{a}, \mathbf{b} \rangle_T = \mathbf{b}^{\top} \mathbf{W}_t \mathbf{a}$:

$$(\mathbf{C} \mathbf{W}_s) \phi_r = \sigma_r^2 \phi_r, \quad (\mathbf{K} \mathbf{W}_t) \psi_r = \sigma_r^2 \psi_r$$

with $\mathbf{C} = \mathbf{D} \mathbf{D}^{\top}$ the spatial covariance and $\mathbf{K} = \mathbf{D}^{\top} \mathbf{D}$ the temporal correlation matrix.

- both problems share the **same** σ_r , so solve whichever is smaller
- $\mathbf{C} \in \mathbb{R}^{n_s \times n_s}$ and $\mathbf{K} \in \mathbb{R}^{n_t \times n_t}$, and usually $n_s \gg n_t$, so the temporal problem wins (*this is Sirovich's method of snapshots, 1987*)

$$\phi_r = \frac{1}{\sigma_r} \mathbf{D} \psi_r$$

one basis follows from the other by projection

POD as an SVD

If $\mathbf{W}_s$ and $\mathbf{W}_t$ are proportional to the identity (uniform sampling in space and time), the POD is the SVD of the dataset matrix:

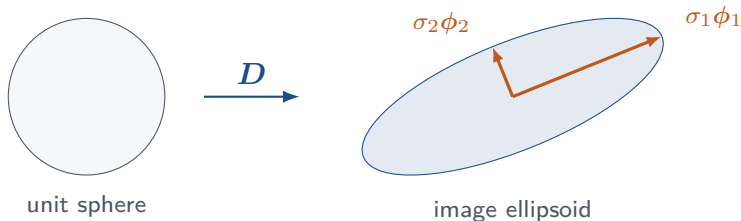
$$\mathbf{D} = \mathbf{\Phi} \mathbf{\Sigma} \mathbf{\Psi}^\top \quad \Longleftrightarrow \quad u(x_i, t_k) = \sum_r \sigma_r \phi_r(x_i) \psi_r(t_k)$$

- $\mathbf{\Phi}$ holds the **spatial** structures, the POD modes
- $\mathbf{\Psi}$ holds the **temporal** or parametric structures
- σ_r is the amplitude of each mode, with $\sigma_1 \geq \sigma_2 \geq \dots$
- both sets are orthonormal in their own inner product

(In code this is a thin SVD. For large datasets a randomised SVD can be one to two orders of magnitude cheaper, at a small and measurable loss of accuracy in the trailing singular values.)

What the SVD does geometrically

Any matrix maps the unit sphere onto an ellipsoid. Φ gives the axis *directions*, σ_r the axis *lengths*, and Ψ says which input direction produced each axis.



keeping n_r modes = keeping the n_r longest axes

For snapshot data the ellipsoid is extremely flat, and that flatness is what makes reduction possible.

Optimality

$$\Phi_{n_r} = \arg \min_{\dim=n_r} \sum_{k=1}^{n_t} \|\mathbf{d}_k - P \mathbf{d}_k\|^2 \quad \text{with minimum} \quad \sum_{r>n_r} \sigma_r^2$$

- Eckart–Young: no other n_r -dimensional **linear** space does better
- optimality holds *on this data* and *in this norm*
- the error is known before building any model, since it is the tail of the spectrum

Over to you

The minimum equals $\sum_{r>n_r} \sigma_r^2$: what does that tell you before any model is built?

Optimal representation $\neq$ optimal dynamics.

Energy content and choosing n_r

$$E_{n_r} = \frac{\sum_{r \leq n_r} \sigma_r^2}{\sum_r \sigma_r^2}$$

- a common rule of thumb is $E_{n_r} \geq 99\%$, or a visible knee in the spectrum
- **caution**: energy measures representation, not the error of the reduced *dynamics*
- slow decay is itself a result. It says that no small linear space exists

Produced in Tutorial 3

The singular values on a semilog axis, the cumulative energy with the 99% and 99.9% levels marked, and the first four modes. How many modes does 99.9% ask for, and does the reduced model actually need that many?

The temporal side: what Ψ tells you

$$\mathbf{d}_k \approx \sum_{r \leq n_r} \sigma_r \phi_r \psi_r(t_k), \quad a_r(t_k) = \sigma_r \psi_r(t_k)$$

- $\psi_r(t)$ shows how each structure is *activated in time*
- for parametric data, ψ_r vs μ shows the parameter dependence
- they give the **reference trajectory** against which a reduced model is validated

Produced in Tutorial 3

Plot ψ_1, ψ_2, ψ_3 against time for two training parameter pairs. Which modes carry the transient and which carry the steady response? Keep these curves: in the last part you will compare them against the coefficients your reduced model predicts.

Mean subtraction and lifting

$$u(x, t) = \bar{u}(x) + u'(x, t), \quad \bar{u} = \frac{1}{n_t} \sum_k u(x, t_k)$$

- applied to the fluctuations, the modes describe **dynamics** rather than the mean state
- the mean becomes an affine offset, which produces constant terms in the reduced model
- this often improves E_{n_r} dramatically at the cost of one extra field

The lifting $h(x)$ of Section 3 does the same job *and* enforces the boundary conditions: $u = h + \sum_r a_r \phi_r$.

(If every snapshot satisfies homogeneous boundary conditions, so does every POD mode, being a linear combination of them.)

Which inner product? *(back to Section 2)*

A plain SVD of $\mathbf{D}$ is optimal in the **Euclidean** norm. The physical norm is

$$\|u\|_{L^2}^2 = \int_{\Omega} u^2 \, d\Omega \approx \mathbf{u}^\top \mathbf{W}_s \mathbf{u}$$

- on a uniform grid $\mathbf{W}_s \propto \mathbf{I}$ and the two agree up to a scalar
- on a non-uniform or finite-element mesh they **differ**, and refined regions get over-weighted
- **weighted POD**: SVD of $\mathbf{W}_s^{1/2} \mathbf{D}$, then $\Phi = \mathbf{W}_s^{-1/2} \mathbf{U}$, so that $\Phi^\top \mathbf{W}_s \Phi = \mathbf{I}$

The optimal subspace depends on the norm. Use the one in which your error matters.

POD under other names, and what it is not

turbulence	Proper Orthogonal Decomposition (Lumley 1967)
statistics	Principal Component Analysis
stochastic processes	Karhunen–Loève expansion
linear algebra	truncated SVD / low-rank approximation

- ✓ an optimal **representation space** for the observed data
- ✓ a ranking of structures and an a-priori error estimate
- × **no** equation for how the coefficients evolve
- × no guarantee outside the sampled regime

(Other decompositions target dynamics or scales rather than energy. They belong to other lectures of this course.)

Plug the POD basis into the Galerkin machine

$$u_{n_r}(x, t) = \underbrace{h(x)}_{\text{lifting / mean}} + \sum_{r=1}^{n_r} a_r(t) \phi_r^{\text{POD}}(x), \quad n_r \ll n_s$$

Residual and Galerkin condition – **unchanged from Section 3**:

$$\mathcal{R} = \partial_t u_{n_r} - \mathcal{F}(u_{n_r}; \boldsymbol{\mu}), \quad \langle \mathcal{R}, \phi_r \rangle_{\Omega} = 0$$

Nothing new is happening mathematically.
The only novelty is **where the ϕ_r came from**.

Over to you

Which of the operators assembled in Tutorial 2 can be reused unchanged, and which have to be rebuilt?

What the projection produces, term by term

Start from the **full-order** system for the lifted field $\mathbf{v} = \mathbf{u} - \mathbf{h}$:

$$\dot{\mathbf{v}} = \underbrace{\mathbf{c}_0^{\text{full}}}_{\text{constant}} + \underbrace{\mathbf{L}^{\text{full}}\mathbf{v}}_{\text{linear}} + \underbrace{\mathbf{N}(\mathbf{v})}_{\text{quadratic}}, \quad \mathbf{v} \in \mathbb{R}^{n_s}$$

Substitute $\mathbf{v} = \Phi_{n_r} \mathbf{a}$ and impose $\Phi_{n_r}^\top \mathbf{W}_s \mathcal{R} = \mathbf{0}$:

full order	reduced	size
mass	$\mathbf{A}_{n_r} = \Phi_{n_r}^\top \mathbf{W}_s \Phi_{n_r} = \mathbf{I}$	$n_r \times n_r$
$\mathbf{c}_0^{\text{full}}$	$\mathbf{c}_0 = \Phi_{n_r}^\top \mathbf{W}_s \mathbf{c}_0^{\text{full}}$	n_r
$\mathbf{L}^{\text{full}}$	$\mathbf{L} = \Phi_{n_r}^\top \mathbf{W}_s \mathbf{L}^{\text{full}} \Phi_{n_r}$	$n_r \times n_r$
$\mathbf{N}$	$C_{mnk} = -\langle \phi_m, \phi_n \phi_k' \rangle_\Omega$	$n_r \times n_r \times n_r$

($\mathbf{A}_{n_r} = \mathbf{I}$ only because the modes are orthonormal in the same inner product used to project.)

Generic ROM structure for quadratic PDEs

$$\dot{\mathbf{a}} = \mathbf{c}_0 + \mathbf{L}(\boldsymbol{\mu}) \mathbf{a} + \mathbf{C} : (\mathbf{a} \otimes \mathbf{a}), \quad \mathbf{a}(0) = \boldsymbol{\Phi}_{n_r}^\top \mathbf{W}_s \mathbf{v}(0)$$

For the running problem, reading the three groups off the lifted equation

$$\dot{v} = (h + \alpha f) + (v + \nu v_{xx} - h v_x) - v v_x:$$

reduced object	what it carries
$\mathbf{c}_0 \in \mathbb{R}^{n_r}$	the lifting h and the source αf
$\mathbf{L} \in \mathbb{R}^{n_r \times n_r}$	diffusion νv_{xx} , and the convection by the lifting , $-h v_x$
$\mathbf{C} \in \mathbb{R}^{n_r \times n_r \times n_r}$	the field convecting itself , $-v v_x$

- the split is by **degree in v** , not by physics: zeroth, first, second
- every entry is a number, computed once; the grid appears nowhere in the reduced system

(This is the structure of Galerkin models of the incompressible Navier–Stokes equations, for the same reason: that convective term is quadratic too.)

Affine parameter dependence

If the operators depend affinely on the parameters,

$$\mathbf{L}(\nu) = \nu \mathbf{L}^{\text{diff}} + \mathbf{L}^{\text{conv}}, \quad \mathbf{c}_0(\alpha) = \mathbf{c}_{00} + \alpha \mathbf{f},$$

every μ -independent piece is assembled **once, offline**.

- online, form two small arrays and integrate n_r ODEs
- there is **no re-projection** and the fine grid is never touched
- a non-affine dependence needs an interpolation of the operators instead

Affine structure is the difference between a ROM that is fast and a ROM that is merely small.

Cost accounting and the offline/online split

Offline

1. run the FOM for training μ
2. snapshots $\rightarrow$ lift $\rightarrow D$
3. SVD $\rightarrow \Phi_{n_r}$
4. project $\rightarrow c_0, L, C$

Online

1. receive μ
2. integrate n_r ODEs
3. estimate / predict / optimise
4. reconstruct $u \approx h + \Phi_{n_r} a$ if needed

	FOM	ROM
states	$n_s \sim 10^5\text{--}10^8$	$n_r \sim 5\text{--}50$
cost per step	$\mathcal{O}(n_s)$ or worse	$\mathcal{O}(n_r^3)$
time-step limit	grid stiffness	none in practice

(The break-even number of online queries, rather than the speed-up factor, is what decides whether the offline effort pays off.)

Two errors, again

$$\underbrace{u - u_{n_r}}_{\text{ROM error}} = \underbrace{(u - Pu)}_{\varepsilon_{\text{proj}}: \text{ the space}} + \underbrace{(Pu - u_{n_r})}_{\text{the projected dynamics}}$$

$\varepsilon_{\text{proj}}$	e_{ROM}	diagnosis
small	small	the ROM is fine
small	large	dynamics , a closure or stability problem
large	large	space , the basis cannot represent the solution

Over to you

Write down the two quantities you would compute to fill this table, and say which one needs no reduced model at all.

Why ROMs go unstable

- truncation discards the small, **dissipative** scales
- the retained modes keep receiving energy but can no longer shed it
- the result is drift, spurious growth or blow-up, often late in time

Remedies, in increasing sophistication:

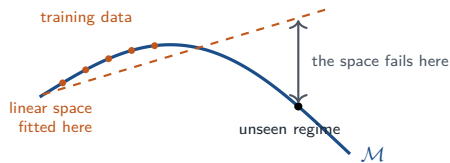
- add modes (*cheap, and not always enough*)
- add an eddy-viscosity or closure term calibrated on the training data
- use a Petrov–Galerkin or energy-preserving projection
- fit c_0, L, C to the data instead of projecting them

(This remains an active research area rather than a solved problem.)

The solution manifold

$$\mathcal{M} = \{u(\cdot, t; \boldsymbol{\mu}) : t \in [0, T], \boldsymbol{\mu} \in \mathcal{P}\}$$

- every solution we could ever need lies on $\mathcal{M}$
- $\mathcal{M}$ is generally **curved**, while $\mathcal{V}_r$ is flat
- POD fits the best flat space to the part of $\mathcal{M}$ we sampled
- far from that part, a flat space is the wrong shape



(The dimension of $\mathcal{M}$ is small, at most the number of parameters plus one for time. The dimension of the smallest linear space containing it is a different matter entirely.)

How well can *any* linear space do?

Kolmogorov (1936) asked exactly this. The n_r -width of a set $\mathcal{M}$ is

$$d_{n_r}(\mathcal{M}) = \inf_{\dim \mathcal{V}_r = n_r} \sup_{u \in \mathcal{M}} \inf_{w \in \mathcal{V}_r} \|u - w\|_{L^2}$$

read from the inside out: approximate u as well as $\mathcal{V}_r$ allows, take the **worst** u on the manifold, then choose the **best** space.

- d_{n_r} is a property of the *problem*, not of any method
- POD is the same idea in a mean-square rather than worst-case sense, so σ_r decay and d_{n_r} decay tell the same story
- **no projection-based reduced model with n_r modes can beat d_{n_r}** , however clever the projection

If d_{n_r} decays slowly, the problem itself refuses linear reduction. That situation is the **Kolmogorov barrier**.

The barrier in practice, and how it is worked around

Where it bites

- advection of a sharp front or a wave
- moving shocks and contact discontinuities
- moving geometries and free surfaces
- high Reynolds number, weak dissipation

A translating step has $d_{n_r} \sim n_r^{-1/2}$: halving the error costs four times the modes.

Ways around it

- **transform first**: shift or freeze the wave, then reduce (*method of freezing, shifted POD, registration*)
- **several local spaces**: cluster the snapshots, switch bases
- **leave linearity**: quadratic manifolds and autoencoders, at the price of a more intrusive projection (*arXiv:2204.02462*)
- **correct what the truncation lost**: learn the closure error in the latent space (*arXiv:2507.00634*)
- **adapt online**: evolve the basis as the solution moves

Diagnose before you cure: a slowly decaying spectrum is not a bug in your code, it is a statement about your problem.

Non-polynomial nonlinearity: hyper-reduction

For a general $N(\cdot)$, the projected term $\Phi_{n_r}^\top N(\Phi_{n_r} \mathbf{a})$ still requires evaluating N at **all** n_s **grid points**.

- the state is small but the right-hand side is not, so the speed-up is lost
- the **Discrete Empirical Interpolation Method** (DEIM) and **gappy POD** evaluate N at $m \ll n_s$ selected points and interpolate it in a second POD basis
- **Energy-Conserving Sampling and Weighting** (ECSW) applies the same idea at the level of elements, selecting a small subset of the mesh and re-weighting it
(*arXiv:2008.02891*)

The incompressible Navier–Stokes convective term is quadratic, so a large part of fluid dynamics escapes this. **The moment the physics stops being polynomial**, through an equation of state, a reaction rate or a turbulence model, the reduction is no longer free.

Why this matters for a digital twin

An asset's operating regime drifts: wear, ageing, faults, new duty cycles, a changing environment. The offline database eventually stops describing the system it is supposed to twin.

What can go wrong

- the state leaves the sampled part of $\mathcal{M}$
- the reduced dynamics drifts, slowly and silently
- the model stays *plausible* while becoming wrong

Cheap health indicators: the residual sampled on the full grid at a few instants, the energy in the last retained mode, the distance of μ from the training set, the mismatch against sensor readings.

What the twin must do

- **monitor** rather than assume validity
- **fall back** to the full model when in doubt
- **update** the basis as data arrives

A reduced model without a validity indicator is an extrapolation, not a twin.

Tutorial 3: purpose

Build the **offline–online** chain end to end on the problem of Tutorial 2, and find out what it costs and where it breaks.

- the FD solver of Tutorial 2 becomes the **full-order model**
- its outputs become the **offline database**
- the Galerkin machinery of Tutorial 2 is reused *verbatim*, and only Φ changes

(This is not a new method. It is the code of Tutorial 2 with a different basis.)

Tutorial 3: assignment

1. **Offline database.** Two **classes** of operating conditions, three runs each:

$$\text{diffusive } \nu = 0.10, \quad \text{convective } \nu = 0.01, \quad \alpha \in \{0.5, 1.0, 1.5\}$$

Lift ($v = u - h$), store about 180 snapshots per run, assemble D . **[implement]**

2. **Basis.** A weighted POD *per class*. Then merge the leading $q = 4$ modes of each class into one **global basis**: orthonormalise the stack, and rank the result by energy. **[implement]**
3. **ROM.** Project onto the global basis and integrate for $n_r = 2, \dots, 8$. **[implement]**
4. **Online.** Predict $(\nu, \alpha) = (0.032, 1.20)$, a pair that was **never simulated**. Report the error, the projection error, and the wall-clock time against the full-order run. **[produce plot]**

Tutorial 3: why two classes

One basis has to serve a **range** of operating conditions, not one point in it.

- a class is a group of conditions sharing a regime: here one diffusive and one convection dominated
- each class is compressed on its own, which is what happens in practice when campaigns arrive at different times
- the leading modes of each are then merged, so no class is outvoted by whichever one carries more energy

Over to you

Modes from two different PODs are each orthonormal. Is their union orthonormal? What has to be done before it can be used as a basis?

Tutorial 3: merging two bases, in two steps [derive in class]

Stack the leading q modes of each class, $\mathbf{S} = [\Phi^{(1)}, \Phi^{(2)}]$. Each half is orthonormal, their union is not:

$$\mathbf{S}^\top \mathbf{W}_s \mathbf{S} = \begin{bmatrix} \mathbf{I}_q & \mathbf{M} \\ \mathbf{M}^\top & \mathbf{I}_q \end{bmatrix}, \quad \mathbf{M} = (\Phi^{(1)})^\top \mathbf{W}_s \Phi^{(2)} \neq \mathbf{0}$$

1. Orthonormalise. A weighted SVD of $\mathbf{W}_s^{1/2} \mathbf{S}$ gives $\mathbf{Q}$, spanning the merged space, with singular values that **measure the overlap**.

2. Rank by energy. Step 1 ordered the directions by how much the classes *agree*, which is not how much data they explain:

$$\mathbf{Z} = \mathbf{Q}^\top \mathbf{W}_s \mathbf{D} = \mathbf{U}_z \Sigma_z \mathbf{V}_z^\top, \quad \Phi_{\text{glob}} = \mathbf{Q} \mathbf{U}_z$$

$\mathbf{U}_z$ is orthogonal, so this is a **rotation inside the same space**: only the ordering changes.

Two SVDs, two questions: **how many** directions are there, then **which** of them matter.

Tutorial 3: what the overlap spectrum measures

The columns of the first SVD are not modes. They are the **sums and differences** of paired modes, and θ_i is the angle between the two spaces:

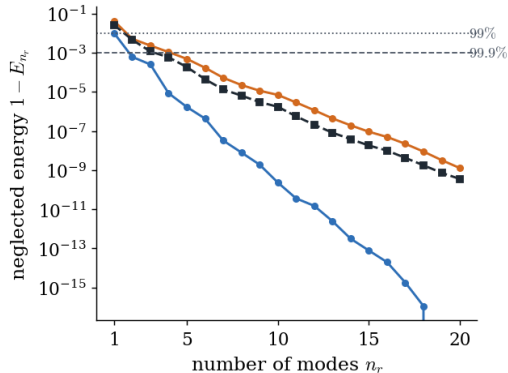
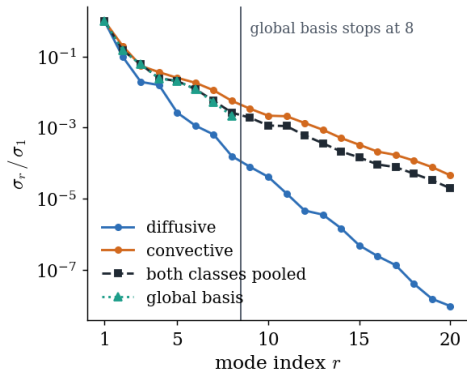
$$\mathbf{u}_i^+ \propto \tilde{\phi}_i^{(1)} + \tilde{\phi}_i^{(2)} \quad (\sigma = \sqrt{1 + \cos \theta_i}), \quad \mathbf{u}_i^- \propto \tilde{\phi}_i^{(1)} - \tilde{\phi}_i^{(2)} \quad (\sigma = \sqrt{1 - \cos \theta_i})$$

So they pair first with last, $\sigma_i^2 + \sigma_{2q+1-i}^2 = 2$. Measured:

pair	σ^+	σ^-	θ	reading
1	1.414	0.003	0.2°	the classes agree on this direction
2	1.414	0.023	1.8°	they agree on this one too
3	1.359	0.392	32°	partly shared
4	1.202	0.745	64°	mostly independent

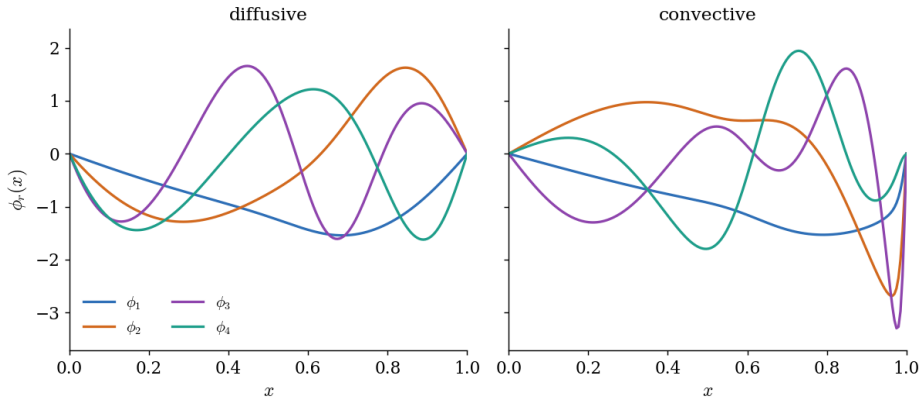
$\sigma^+ = \sqrt{2}$ would mean an exactly shared direction and $\sigma^+ = 1$ a direction only one class has. Dropping a small σ^- removes a **difference**, never a mode: both members survive, to within $\theta/2$. Here nothing is dropped, because the tolerance is a threshold on the angle, $\theta < 2 \text{ tol}$, and $\text{tol} = 10^{-10}$.

Tutorial 3: the spectra



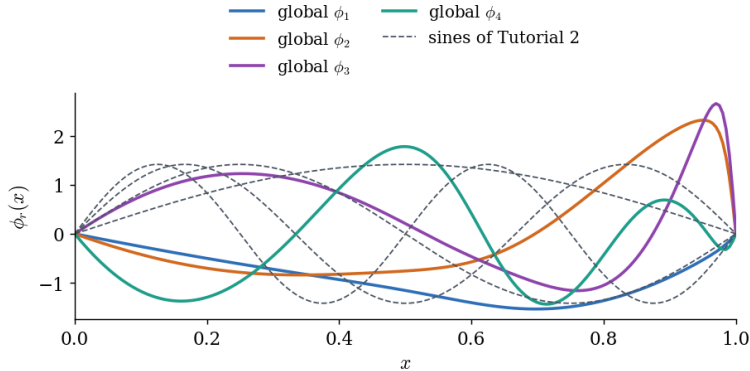
The convective class needs more modes than the diffusive one for the same energy, and the pooled spectrum sits between them. The global basis, built from four modes per class, tracks the pooled spectrum until it runs out of directions at $n_r = 8$.

Tutorial 3: the modes of each class



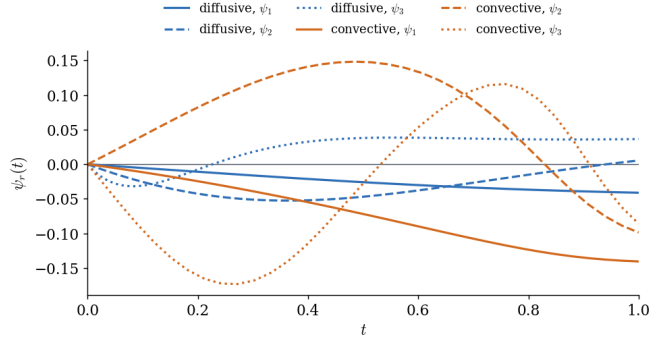
Same four leading modes, same colours, one panel per regime, on a shared vertical axis. ϕ_1 is nearly the same curve in both. The higher modes are not: the convective ones pile their structure against $x = 1$, where the front stands, and are much larger there.

Tutorial 3: the modes the data chooses



The POD modes put their structure where the source sits and where the front forms. The sine modes of Tutorial 2 spread it uniformly, because they were chosen before the problem was looked at.

Tutorial 3: the temporal structures



$\psi_r(t)$ for the first training run of each class, colour by class and line style by r . ψ_1 moves monotonically and settles: it carries the drift towards the steady state. The higher ones change sign and decay, so they carry the **transient**, which is why a two-mode model is at its worst early and recovers later. (*the sign of a POD mode is arbitrary; only $|\psi_r|$ and the relative signs mean anything*)

Tutorial 3: the reduced model

$$\dot{\mathbf{a}} = \mathbf{c}_r(\alpha) + \mathbf{L}_r(\nu) \mathbf{a} + \mathbf{C}_r : (\mathbf{a} \otimes \mathbf{a}), \quad \mathbf{a}(0) = \mathbf{0}$$

with, and this is the whole point,

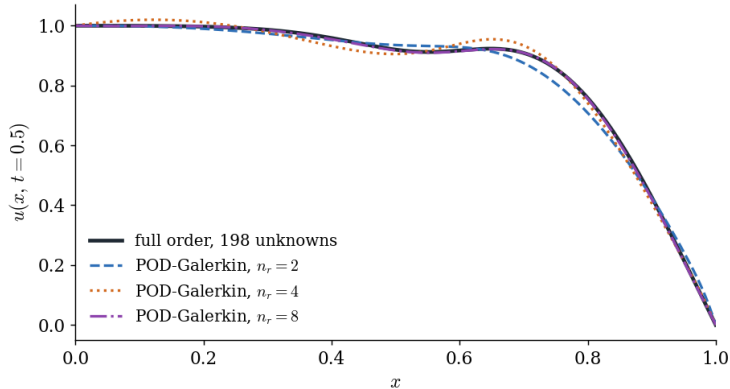
$$\mathbf{c}_r(\alpha) = \mathbf{g}_r + \alpha \mathbf{f}_r, \quad \mathbf{L}_r(\nu) = \mathbf{A}_r + \nu \mathbf{B}_r, \quad \mathbf{C}_r \text{ parameter free}$$

- every integral is evaluated **once**, offline
- a new μ then costs a scalar multiplication and an addition
- $\Phi^\top \mathbf{W}_s \Phi = \mathbf{I}$, so the reduced mass matrix is the identity

Over to you

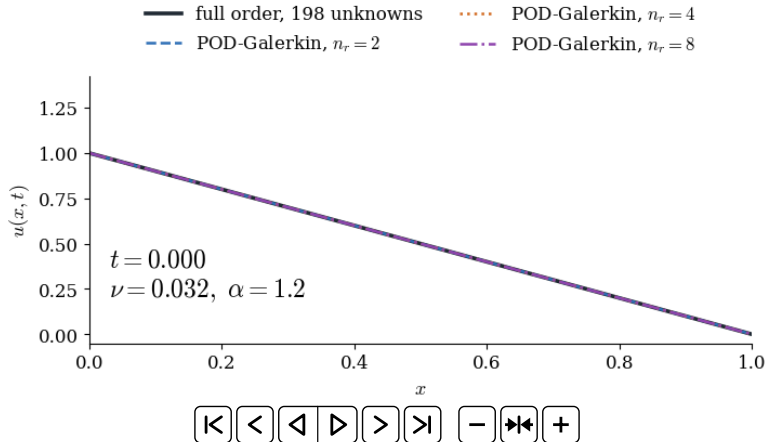
Which of the five stored objects would stop being reusable if the source position x_f became a parameter as well?

Tutorial 3: prediction at an unseen parameter



At $\nu = 0.032$, $\alpha = 1.2$, which is in neither class. Two modes miss the front, four place it, eight are indistinguishable from a model with 198 unknowns.

Tutorial 3: the same prediction, in time

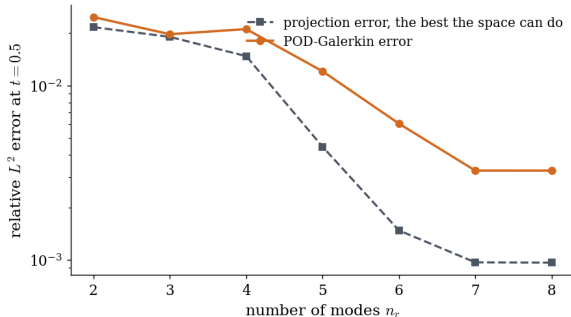


The reduced solutions are integrated with a step thirty times larger than the full-order one, because the stiff part is now a matrix of size n_r and is treated implicitly.

Tutorial 3: two errors, separated

Both curves measure $\|u - \tilde{u}\|/\|u\|$ at $t = 0.5$. They differ in the coefficients:

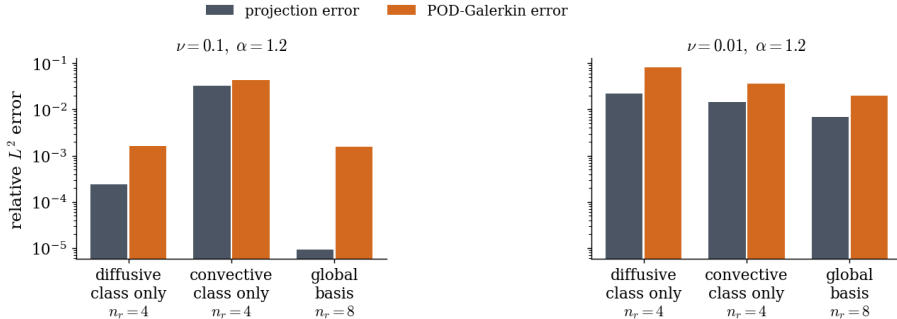
curve	$\tilde{u}$	coefficients from
projection , $\varepsilon_{\text{proj}}$	$h + \Phi_{n_r} \mathbf{a}^{\text{proj}}$	$\mathbf{a}^{\text{proj}} = \Phi_{n_r}^\top \mathbf{W}_s(\mathbf{u} - \mathbf{h})$, the known solution
POD-Galerkin , e_{ROM}	$h + \Phi_{n_r} \mathbf{a}(t)$	integrating $\dot{\mathbf{a}} = \mathbf{c}_0 + \mathbf{L}\mathbf{a} + \mathbf{C} : (\mathbf{a} \otimes \mathbf{a})$



- the dashed curve is the **best the space can do**, and needs no model at all
- the gap above it is the error of the **dynamics**
- both flatten once n_r reaches the size of the global basis

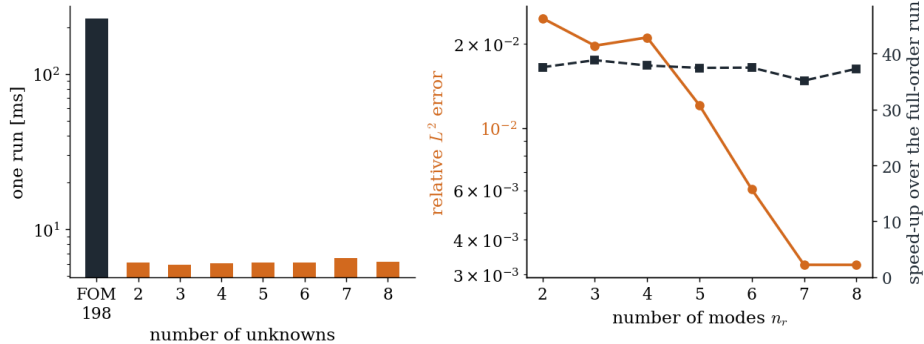
A better time integrator moves the solid curve only. A better basis moves both.

Tutorial 3: why the basis has to be global



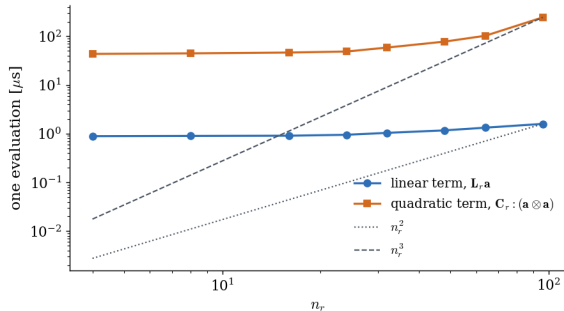
Each basis at its natural size: four modes for a single class, eight for the two merged. A single-class basis is excellent at home and one to two orders of magnitude worse abroad. The global basis is usable at both points, and that is what it is bought for.

Tutorial 3: the cost



Left: one run, full order against reduced. Right: the same numbers as an error-against-time trade-off. About $35\times$ faster at $n_r = 8$, with a relative error of $3 \cdot 10^{-3}$. The offline stage cost six full-order runs, so the model pays for itself after roughly ten queries.

Tutorial 3: where the online time goes



- the linear term costs $O(n_r^2)$
- the quadratic contraction costs $O(n_r^3)$ and dominates everything else
- below $n_r \approx 30$ both are flat: that plateau is interpreter overhead, not arithmetic

$O(n_r^3)$ is why hyper-reduction exists.

Tutorial 3: questions to answer with your results

On the space and on the dynamics

Which of the two error curves would a smaller time step improve, and which one would only a larger basis improve? Use the extrapolation case $\nu = 0.005$ to decide which of the two is failing there.

On the cost

Eight modes replace 198 unknowns, yet the speed-up is $35\times$ and not 25 times that. Where does the rest go? Count the operations per step, and count the steps.

Tutorial 3: debugging checklist

- is $\Phi^T \mathbf{W}_s \Phi = \mathbf{I}$? (*the weighting trap*)
- does $\mathbf{a}(0) = \mathbf{0}$ reconstruct $u(x, 0) = 1 - x$?
- project a *training* trajectory and compare $\mathbf{a}_{\text{ROM}}(t)$ against $\mathbf{a}_{\text{proj}}(t)$ (*if they diverge, the dynamics is at fault and not the space*)
- is $\mathbf{C}_r$ built with the same derivative operator as the full model?
- if it blows up, increase n_r first, then check the sign of $\mathbf{C}_r$
- always compare $h + \Phi_{n_r} \mathbf{a}$ against u , never $\mathbf{a}$ against u

POD is optimal for the data it was given. **It is not universally optimal.**

Section 5

Synthesis

The one equation of today

$$\underbrace{\text{choose a finite-dimensional space}}_{\substack{\text{basis } \{\phi_n\} \\ \text{finite-dimensional dynamics for } \mathbf{a}(t)}} + \underbrace{\text{project the governing equation}}_{\langle \mathcal{R}, \phi_m \rangle_{\Omega} = 0} =$$

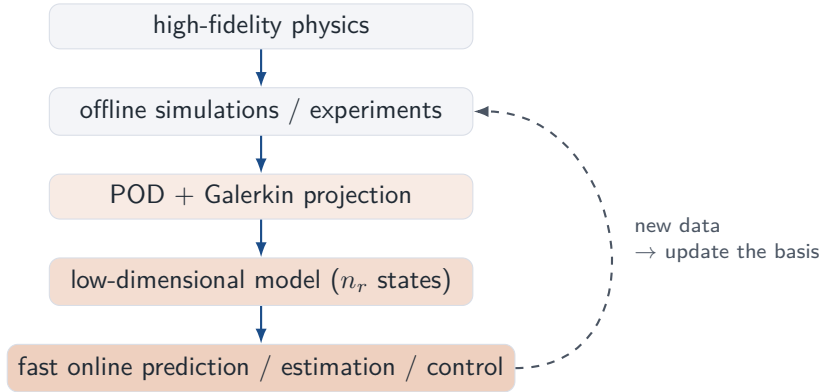
Spectral global, analytically prescribed basis

FEM local, mesh-based prescribed basis

POD global, **data-adapted** basis

FD a different philosophy, discretising the operator

The digital-twin chain



(The dashed feedback arrow is the subject of the following lectures.)

Take-home messages

POD-Galerkin is **not** new machinery. It is Galerkin projection on a low-dimensional space *learned from representative solution data*.

Spend the computational effort once, reuse previously generated information, and obtain models cheap enough to evaluate in real time.

And, in front of any reduced model, always ask:
is my **space** adequate, or is my **projection** failing?

References

Approximation and projection

1. L. N. Trefethen, *Spectral Methods in MATLAB*, SIAM, 2000.
2. J. P. Boyd, *Chebyshev and Fourier Spectral Methods*, 2nd ed., Dover, 2001.
3. T. J. R. Hughes, *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*, Dover, 2000.
4. A. N. Brooks, T. J. R. Hughes, Streamline upwind/Petrov–Galerkin formulations for convection dominated flows, *Comput. Methods Appl. Mech. Eng.* 32, 199–259, 1982.

References

POD and modal analysis

5. L. Sirovich, Turbulence and the dynamics of coherent structures, *Quart. Appl. Math.* 45(3), 561–571, 1987.
6. G. Berkooz, P. Holmes, J. L. Lumley, The proper orthogonal decomposition in the analysis of turbulent flows, *Annu. Rev. Fluid Mech.* 25, 539–575, 1993.
7. K. Taira *et al.*, Modal analysis of fluid flows: an overview, *AIAA Journal* 55(12), 4013–4041, 2017.
8. S. Dawson, The proper orthogonal decomposition, in *Data-Driven Fluid Mechanics: Combining First Principles and Machine Learning*, M. A. Mendez, A. Ianiro, B. R. Noack, S. L. Brunton (eds.), Cambridge University Press, 117–132, 2023.
9. M. A. Mendez, Generalized and multiscale modal analysis, in *Data-Driven Fluid Mechanics: Combining First Principles and Machine Learning*, M. A. Mendez, A. Ianiro, B. R. Noack, S. L. Brunton (eds.), Cambridge University Press, 153–181, 2023. [arXiv:2208.12630](#)
10. I. Tirelli, M. A. Mendez, A. Ianiro, S. Discetti, A meshless method to compute the proper orthogonal decomposition and its variants from scattered data, *Proc. R. Soc. A* 481(2313), 20240526, 2025. [arXiv:2407.03173](#)
11. M. Ratz, A. Parente, M. A. Mendez, Meshless data-driven decompositions with RBF-based inner products, *Computers and Fluids* 315, 107133, 2026. [arXiv:2511.03264](#)
12. R. Poletti, L. Schena, D. Ninni, M. A. Mendez, MODULO: a Python toolbox for data-driven modal decomposition, *J. Open Source Software* 9(102), 6753, 2024. [doi:10.21105/joss.06753](#)

References

Projection-based reduced-order models

13. P. Benner, S. Gugercin, K. Willcox, A survey of projection-based model reduction methods for parametric dynamical systems, *SIAM Review* 57(4), 483–531, 2015.
14. C. W. Rowley, S. T. M. Dawson, Model reduction for flow analysis and control, *Annu. Rev. Fluid Mech.* 49, 387–417, 2017.
15. A. Quarteroni, A. Manzoni, F. Negri, *Reduced Basis Methods for Partial Differential Equations*, Springer, 2016.
16. S. Chaturantabut, D. C. Sorensen, Nonlinear model reduction via discrete empirical interpolation, *SIAM J. Sci. Comput.* 32(5), 2737–2764, 2010.
17. S. Grimberg, C. Farhat, R. Tezaur, C. Rabinovich, Mesh sampling and weighting for the hyperreduction of nonlinear Petrov–Galerkin reduced-order models with local reduced-order bases, *Int. J. Numer. Methods Eng.* 122(7), 1846–1874, 2021. [arXiv:2008.02891](https://arxiv.org/abs/2008.02891)

References

Limits of linear reduction, and ways around them

18. M. Ohlberger, S. Rave, Reduced basis methods: success, limitations and future challenges, *Proc. ALGORITHMY*, 1–12, 2016.
19. C. Greif, K. Urban, Decay of the Kolmogorov N -width for wave problems, *Appl. Math. Letters* 96, 216–222, 2019.
20. K. Lee, K. T. Carlberg, Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders, *J. Comput. Phys.* 404, 108973, 2020.
21. J. Barnett, C. Farhat, Quadratic approximation manifold for mitigating the Kolmogorov barrier in nonlinear projection-based model order reduction, *J. Comput. Phys.* 464, 111348, 2022.
[arXiv:2204.02462](#)
22. S. Ares de Parga, R. Tezaur, C. G. Hernández, C. Farhat, Nonlinear projection-based model order reduction with machine learning regression for closure error modeling in the latent space, 2025.
[arXiv:2507.00634](#)